Online Food Delivery Management System

INDEX

s.no	Title	Page number
1.	Introduction	2
2.	Problem Statement	3
3.	Objectives of the Project	4
4.	Scope of the Project	5
5.	Technologies Used	6
6.	System Architecture	7
7.	APIs & Business logic	8-9
8.	Implementation & Output	10
	8.1. Layer and Project of Spring Boot	10
	8.2. Customer	10-21
	8.3. Restaurant	22-32
	8.4. Menu Item	33-41
	8.5. order	42-56
	8.6. Order Item	57-63
	8.7. Payment	64-69
9.	Future Enhancements	70-74
10.	Summary	75

1. Introduction

FoodieHub is a modern Online Food Delivery Management System developed to simplify the process of ordering food from multiple restaurants through a centralized digital platform. In today's fast-paced world, customers prefer online services that save time and provide convenience. Traditional restaurant ordering systems often involve manual processes such as phone calls, paper bills, delayed communication, and inefficient order tracking. FoodieHub solves these issues by offering a complete backend system where customers can browse restaurants, view menu items, place orders, make payments, and track order status.

The project is developed using Java Spring Boot and follows REST API architecture. It provides separate modules for Customer Management, Restaurant Management, Menu Item Management, Order Processing, Order Item Handling, and Payment Management. Every module is connected with real-world business logic to ensure accuracy and reliability.

For example, customers cannot register with duplicate email or contact numbers. Orders cannot be placed without valid customers, available menu items, and successful payments. Order totals are automatically calculated based on quantity and item price. Restaurants can manage menus efficiently, while administrators can monitor all activities.

FoodieHub demonstrates how enterprise-level backend systems are built using object-oriented programming, relational databases, layered architecture, validations, and API design. This project is highly useful for students and freshers because it represents real industry scenarios such as e-commerce platforms, food delivery apps, and service marketplaces.

The system can be extended in the future with mobile applications, real-time notifications, GPS delivery tracking, user authentication, reviews, and analytics dashboards.

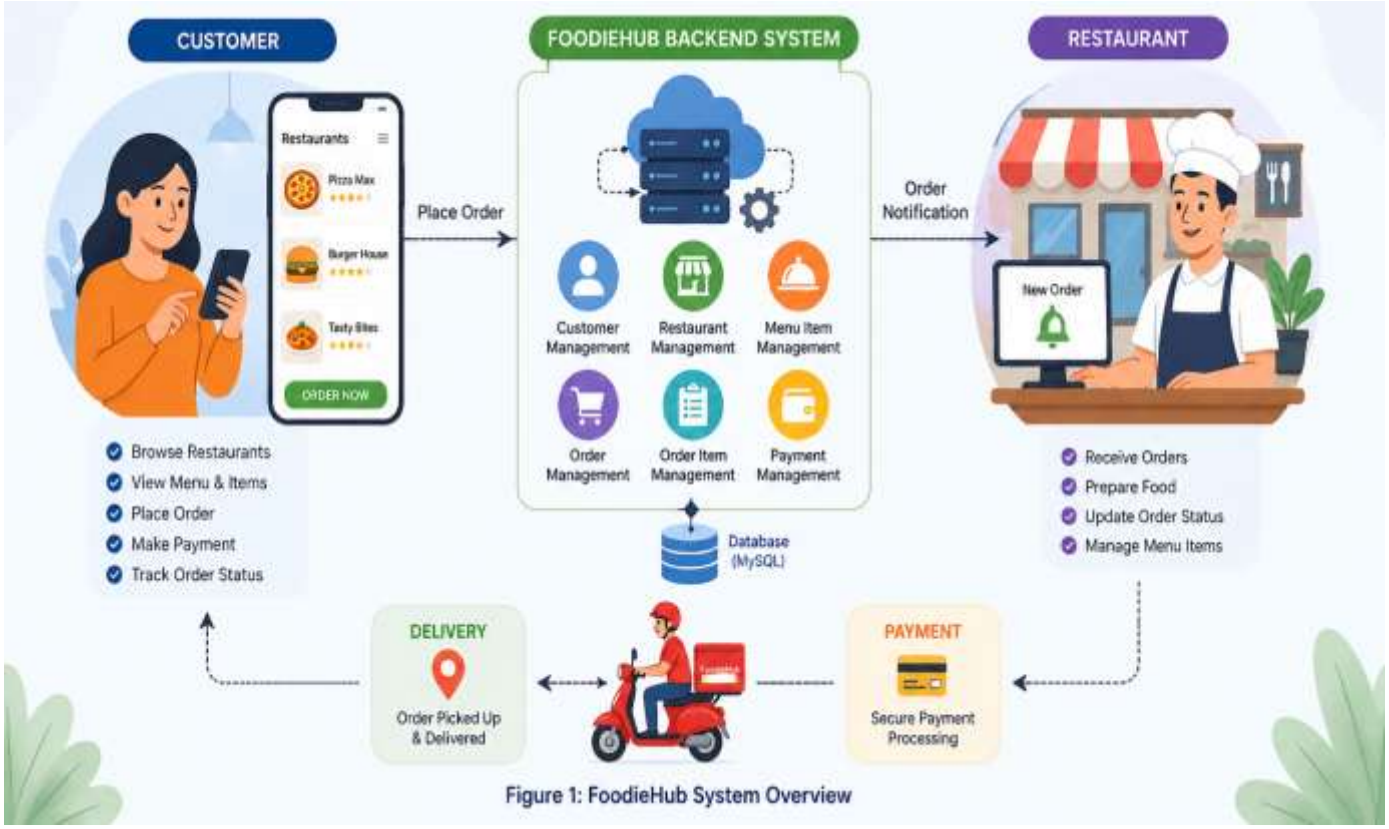


Figure 1: FoodieHub System Overview

2. Problem Statement

Many restaurants and local food businesses still depend on manual order management methods such as phone bookings, handwritten bills, and direct cash transactions. These traditional systems create several operational problems for both customers and restaurant owners.

Customers face issues such as delayed responses, lack of menu visibility, incorrect orders, no order tracking, and payment inconvenience. They may not know which restaurants are available nearby or what menu items are currently in stock. This results in poor customer satisfaction.

Restaurants face challenges like managing multiple customer orders, handling inventory manually, maintaining payment records, calculating totals, and updating order statuses. Human errors in bill calculation, duplicate entries, missed orders, and delayed preparation are common problems.

There is also a lack of centralized data management. Customer details, restaurant data, menu information, and payment history are often stored separately or manually, making reporting and monitoring difficult.

To overcome these problems, there is a need for a digital backend system that can:

- Manage customers efficiently
- Store restaurant and menu data centrally
- Allow easy food ordering
- Calculate totals automatically
- Process payments accurately
- Update order statuses instantly
- Support filtering, sorting, and searching data
- Ensure data integrity using validations

FoodieHub is designed to solve all these issues through a RESTful API-based architecture. It creates a reliable bridge between customers and restaurants while reducing manual workload and improving speed, transparency, and efficiency.

3. Objectives of the Project

The primary objective of FoodieHub is to develop a scalable and efficient Online Food Delivery Management System using Java Spring Boot and MySQL. The project aims to automate restaurant ordering operations and provide a seamless experience for customers and administrators.

Main Objectives:

1. Customer Management

To create APIs that allow adding, updating, deleting, and retrieving customer details with validation rules such as unique email and contact number.

2. Restaurant Management

To maintain restaurant records including name, location, and menu details. Users should be able to search restaurants by location or name.

3. Menu Item Management

To add and manage food items for each restaurant with price control and availability status.

4. Order Processing

To allow customers to place food orders with multiple items. The system should automatically calculate the total bill.

5. Payment Management

To manage payment records and ensure payment amount matches the order total amount.

6. Order Tracking

To update and monitor statuses such as Pending, Preparing, Delivered, and Cancelled.

7. Business Validation

To implement real-world rules like:

- Order must contain at least one item
- Quantity must be minimum one
- Unavailable items cannot be ordered
- Orders can be cancelled only before preparation

8. Search and Pagination

To support sorting, filtering, and pagination for large restaurant and order datasets.

9. Learning Objective

To gain practical knowledge of Spring Boot, JPA, Hibernate, REST APIs, SQL relationships, and layered architecture.

4. Scope of the Project

FoodieHub has a wide scope in the food delivery and restaurant management domain. It can be used by restaurants, cloud kitchens, startups, cafeterias, and local food businesses that want to digitize their order management system.

Future Scope:

- Mobile App Integration
- Real-time Delivery Tracking
- OTP Login
- Ratings and Reviews
- Coupons and Discounts
- Admin Dashboard
- Cloud Deployment
- Microservices Architecture
- AI-based Food Recommendations

Thus, FoodieHub has both academic and commercial relevance.

Figure 2 : ***Scope of the Project***

5. Technologies Used

FoodieHub is built using modern backend development technologies that are widely used in the software industry.

a. **Java**
   Java is the core programming language used to build the project. It provides platform independence, security, object-oriented features, and strong community support.

b. **Spring Boot**
   Spring Boot is used to develop REST APIs quickly with minimal configuration. It simplifies backend development through dependency injection and embedded servers.

c. **Spring Data JPA**
   Used for database interaction. It reduces SQL boilerplate code and allows repository-based CRUD operations.

d. **Hibernate**
   Hibernate acts as the ORM tool that maps Java classes with database tables.

e. **MySQL**
   Used as the relational database for storing customer, restaurant, menu, order, and payment data.

f. **Maven**
   Used for dependency management and project build automation.

g. **Postman**
   Used for API testing and sending HTTP requests.

h. **IDE**
   IntelliJ IDEA / Eclipse / Spring Tool Suite for coding and debugging.

6. System Architecture

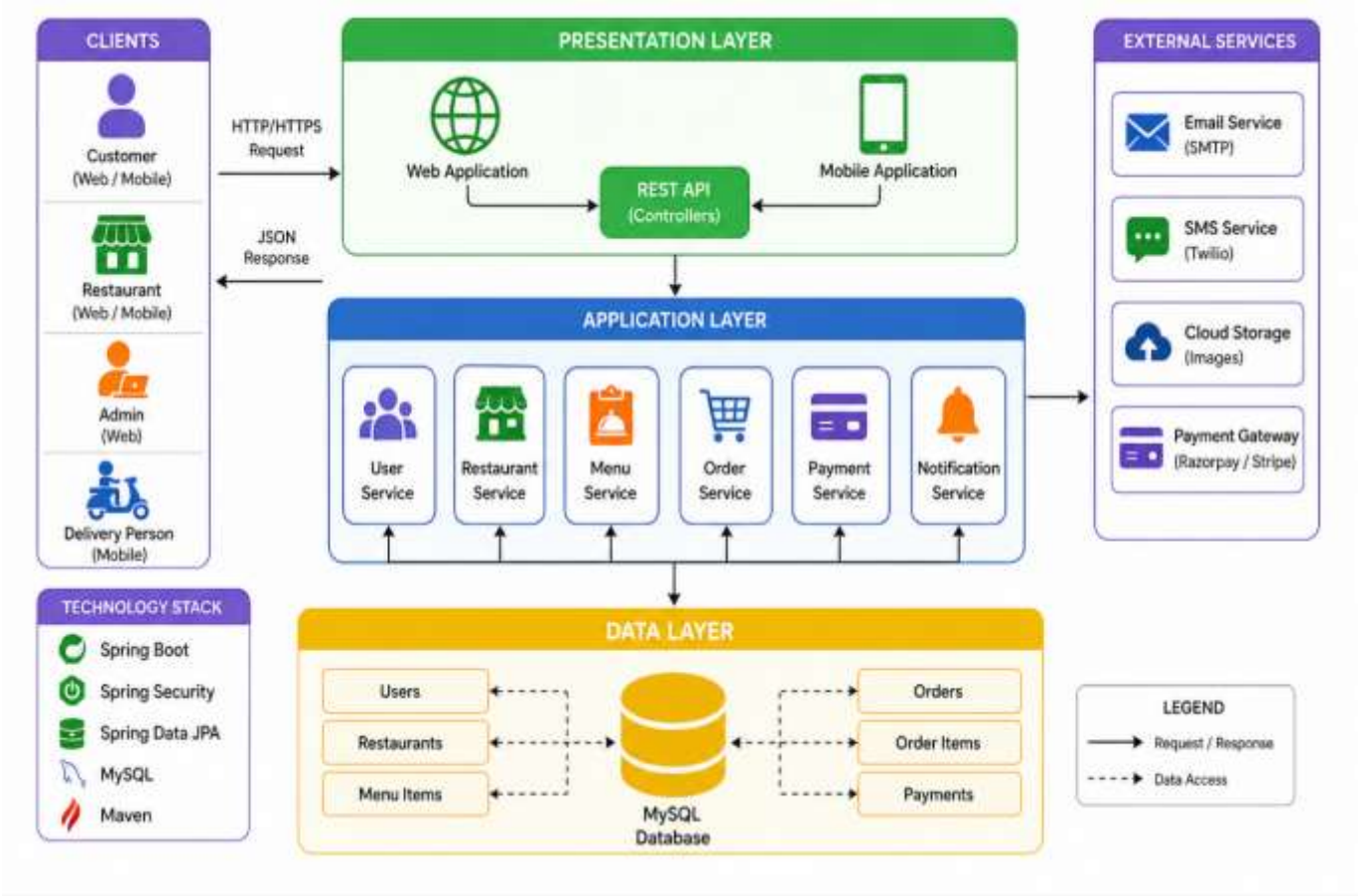


Figure 3 : ***System Architecture***

In above Figure 3 shows the layered architecture of the **FoodieHub – Online Food Delivery Management System**. The system is divided into different layers for better organization and smooth performance.

- **Client Layer:** Includes Customer, Restaurant, Admin, and Delivery Person who access the system through web or mobile applications.
- **Presentation Layer:** Handles user requests through REST APIs and sends responses.
- **Application Layer:** Contains business logic such as customer management, restaurant management, menu handling, order processing, payment, and notifications.
- **Data Layer:** Stores all data in the MySQL database such as customers, restaurants, menu items, orders, and payments.
- **External Services:** Includes Email, SMS, Cloud Storage, and Payment Gateway for additional features.

7. APIs & Business logic

customer

1) create customer
2) get all customers
3) get by id
4) update customer
5) delete customer
6) get by contact

Restaurant

1) add restaurant
2) get all restaurant
3) get restaurant by id
4) update restaurant
5) delete restaurant
6) get restaurant by location
7) get by name
8) get menu item of restaurant
9) get restaurant by pagination or sorting

Menu item

1) add menu items
2) get all menu items
3) get by id
4) update price
5) delete menu item
6) get item greater than a value
7) get item by name

Order

1) place order
2) get all order
3) get by id
4) get order of a customer
5) update order status
6) cancel order
7) get order by status
8) get order by date
9) get order between total amount range
10) get orders of a partial restaurant

Order Item

1) add item to existing order
2) update item quantity
3) remove item from order item
4) get order item of an order

Payment

1) get payment by id
2) get payment by status
3) get payment by payment method
4) update payment status

Business logic

customer

1) email and contact must be unique and contact number must be 10 digit

menu item

1) price cannot be negative
2) menu item cannot be orders if it is unavailable
3) menu item cannot be save if restaurant is not there

order

1) total amount should be calculated based on number of item
2) order can be cancel only before preparation
3) order must have at least one item
4) order cannot be saved if payment is not there, customer is not there or order item is not there

order item

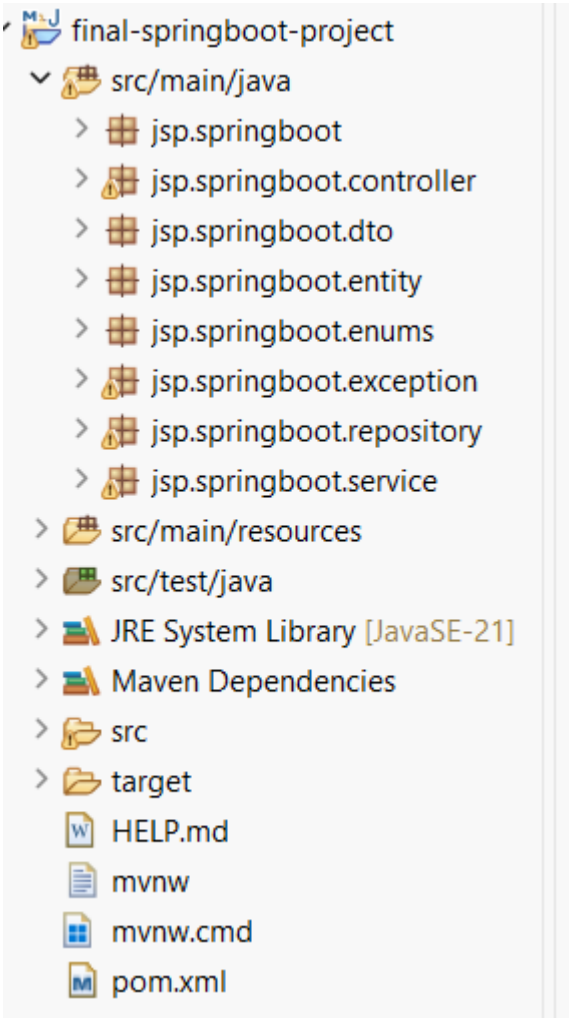
1) quantity must be minimum one
2) subtotal is equal is price * quality

payment

1) payment amount must be same as order total amount

8. Implementation & Output

8.1 Layer and Project of Spring Boot



8.2 Customer :

Application.java

```
package jsp.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

}
```

Customer.java

```java
package jsp.springboot.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;

@Entity
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer customerId;

    @Column(nullable = false)
    private String customerName;

    @Column(unique = true)
    private String email;

    @Column(unique = true)
    private String contact;
    private String address;


    @OneToMany(mappedBy = "customer")
    @JsonIgnore
    private List<Order> orders;

    // getter and setter
}
```

CustomerRepository.java

```java
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import jsp.springboot.entity.Customer;

public interface CustomerRepository extends JpaRepository<Customer, Integer> {

    // 1. create customer
    boolean existsByEmail(String email);
    boolean existsByContact(String contact);

    // 6. get customer by contact
    Optional<Customer> findByContact(String contact);
}
```

ResponseStructure.java

```java
package jsp.springboot.dto;

public class ResponseStructure<C> {
    private Integer statusCode;
    private String message;
    private C data;

    // getter and setter
}
```

GlobalExceptionHandler.java

```java
package jsp.springboot.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import jsp.springboot.dto.ResponseStructure;

@ControllerAdvice
public class GlobalExceptionHandler {

    // ResourceNotFoundException
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ResponseStructure<String>> handleResourceNotFoundException(ResourceNotFoundException exception) {

        ResponseStructure<String> response = new ResponseStructure<>();

        response.setStatusCode(HttpStatus.NOT_FOUND.value());
        response.setMessage(exception.getMessage());
        response.setData("Failure");

        return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.NOT_FOUND);
    }
```

<u>ResourceNotFoundException.java</u>

```java
package jsp.springboot.exception;

public class ResourceNotFoundException extends RuntimeException{
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

GlobalExceptionHandler.java

```java
// InvalidInputException
@ExceptionHandler(InvalidInputException.class)
public ResponseEntity<ResponseStructure<String>> handleInvalidInputException(InvalidInputException exception) {

    ResponseStructure<String> response = new ResponseStructure<>();

    response.setStatusCode(HttpStatus.BAD_REQUEST.value());
    response.setMessage(exception.getMessage());
    response.setData("Failure");

    return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.BAD_REQUEST);
}
```

InvalidInputException.java

```java
package jsp.springboot.exception;

public class InvalidInputException extends RuntimeException{
    public InvalidInputException(String message) {
        super(message);
    }
}
```

GlobalExceptionHandler.java

```java
// BusinessValidationException
@ExceptionHandler(BusinessValidationException.class)
public ResponseEntity<ResponseStructure<String>> handleBusinessValidationException(BusinessValidationException exception) {

    ResponseStructure<String> response = new ResponseStructure<>();

    response.setStatusCode(HttpStatus.BAD_REQUEST.value());
    response.setMessage(exception.getMessage());
    response.setData("Failure");

    return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.BAD_REQUEST);
}
```

BusinessValidationException.java

```java
package jsp.springboot.exception;

public class BusinessValidationException extends RuntimeException{
    public BusinessValidationException(String message) {
        super(message);
    }
}
```

CustomerService.java

```java
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Customer;
import jsp.springboot.exception.BusinessValidationException;
import jsp.springboot.exception.InvalidInputException;
import jsp.springboot.exception.ResourceNotFoundException;
import jsp.springboot.repository.CustomerRepository;

@Service
public class CustomerService {
    @Autowired
    private CustomerRepository repository;

    // 1. create customer
    public ResponseEntity<ResponseStructure<Customer>> createCustomer(Customer customer) {
        if(customer.getContact().length()!=10)
            throw new InvalidInputException("Contact must be 10 digits");

        if(repository.existsByEmail(customer.getEmail()))
            throw new BusinessValidationException("Email already exists");

        if(repository.existsByContact(customer.getContact()))
            throw new BusinessValidationException("Contact already exists");

        ResponseStructure<Customer> response = new ResponseStructure<Customer>();
        response.setStatusCode(HttpStatus.CREATED.value());
        response.setMessage("Create customer Successfully!");
        response.setData(repository.save(customer));

        return new ResponseEntity<ResponseStructure<Customer>>(response, HttpStatus.CREATED);
    }
```

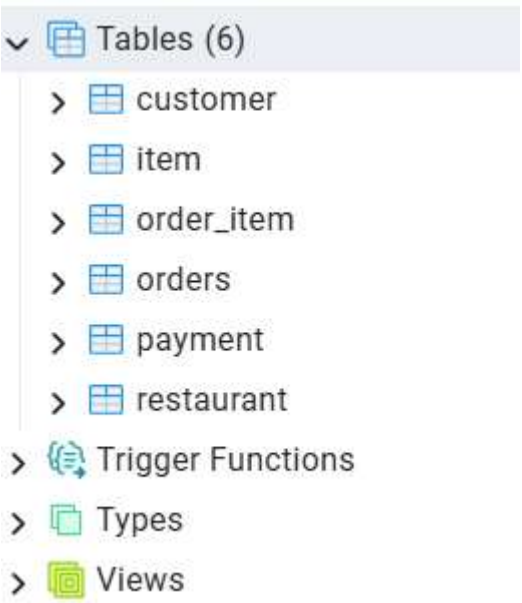
CustomerController.java

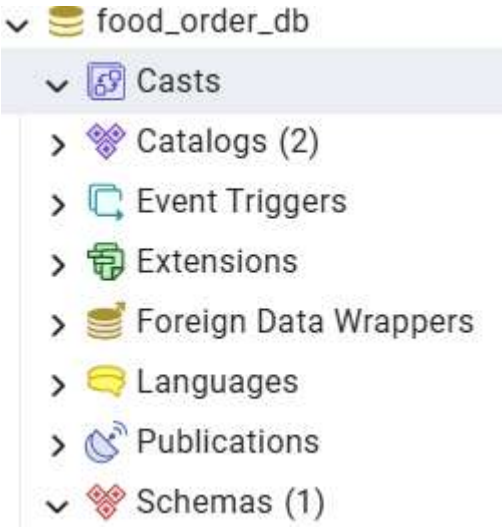
```java
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.RequestEntity;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Customer;
import jsp.springboot.service.CustomerService;

@RestController
@RequestMapping("/customers")
public class CustomerController {
    @Autowired
    private CustomerService service;

    // 1. Create Customer
    @PostMapping
    public ResponseEntity<ResponseStructure<Customer>> createCustomer(@RequestBody Customer customer) {
        return service.createCustomer(customer);
    }
```

Application.properties

```
spring.application.name=final-springboot-project

spring.datasource.url=jdbc:postgresql://localhost:5432/food_order_db
spring.datasource.username=postgres
spring.datasource.password=SUMAN

spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.PostgreSQLDialect
```

pgAdmin





Postman(Output) :

POST http://localhost:8080/customers Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1  {
2    "customerName": "Preeti",
3    "email": "preeti123@gmail.com",
4    "contact": "9879543218",
5    "address": "Udupi, Karnataka"
6  }
```

Body 201 Created 24 ms 360 B

{} JSON Preview Visualize

```
1  {
2      "data": {
3          "address": "Udupi, Karnataka",
4          "contact": "9879543218",
5          "customerId": 11,
6          "customerName": "Preeti",
7          "email": "preeti123@gmail.com"
8      },
9      "message": "Create customer Successfully!",
10     "statusCode": 201
11 }
```

pgAdmin(Output) :

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

	customer_id [PK] integer	address character varying (255)	contact character varying (255)	customer_name character varying (255)	email character varying (255)
1	1	Belgaum, Karnataka	9876543210	Ganesh	ganesh1@example.com
2	2	Bangalore, Karnataka	9876543211	Rahul	rahul2@example.com
3	3	Mysore, Karnataka	9876543212	Sneha	sneha3@example.com
4	4	Hubli, Karnataka	9876543213	Priya	priya4@example.com
5	5	Mangalore, Karnataka	9876543214	Arun	arun5@example.com
6	6	Dharwad, Karnataka	9876543215	Kiran	kiran6@example.com
7	7	Tumkur, Karnataka	9876543216	Pooja	pooja7@example.com
8	8	Bijapur, Karnataka	9876543217	Vikas	vikas8@example.com
9	9	Shimoga, Karnataka	9876543218	Deepa	deepa9@example.com
10	10	Udupi, Karnataka	9876543219	Nikhil	nikhil10@example.com
11	11	Udupi, Karnataka	9879543218	Preeti	preeti123@gmail.com

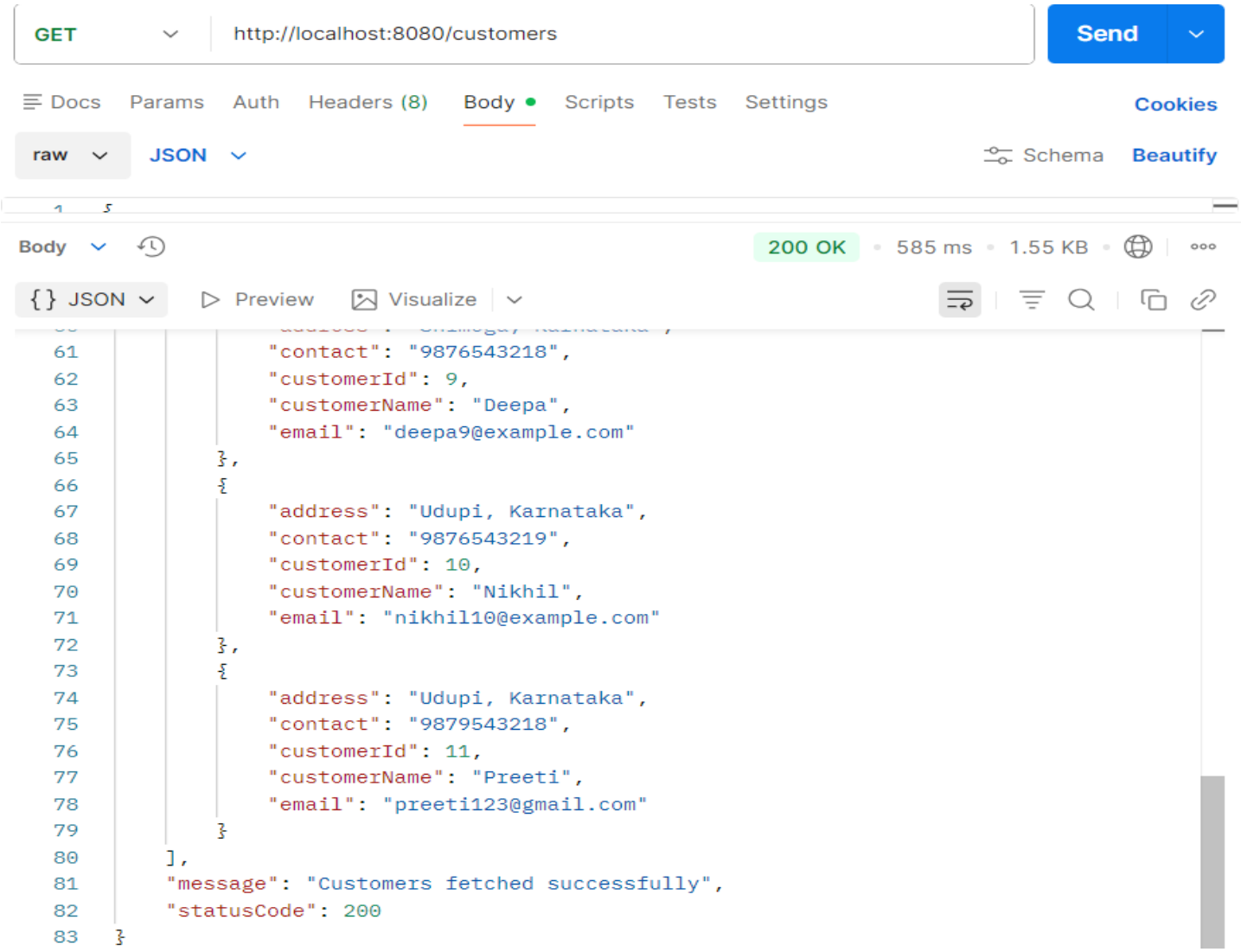
2. get all customers(CustomerService.java)

```java
// 2. get all customers
public ResponseEntity<ResponseStructure<List<Customer>>> getAllCustomers() {
    List<Customer> customers = repository.findAll();
    if(!customers.isEmpty()) {
        ResponseStructure<List<Customer>> response = new ResponseStructure<>();
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Customers fetched successfully");
        response.setData(customers);
        return new ResponseEntity<ResponseStructure<List<Customer>>>(response, HttpStatus.OK);
    }
    throw new ResourceNotFoundException("No Customers available in the DB");
}
```

CustomerController.java

```java
// 2. get all customers
@GetMapping
public ResponseEntity<ResponseStructure<List<Customer>>> getAllCustomers() {
    return service.getAllCustomers();
}
```

Output (Postman):



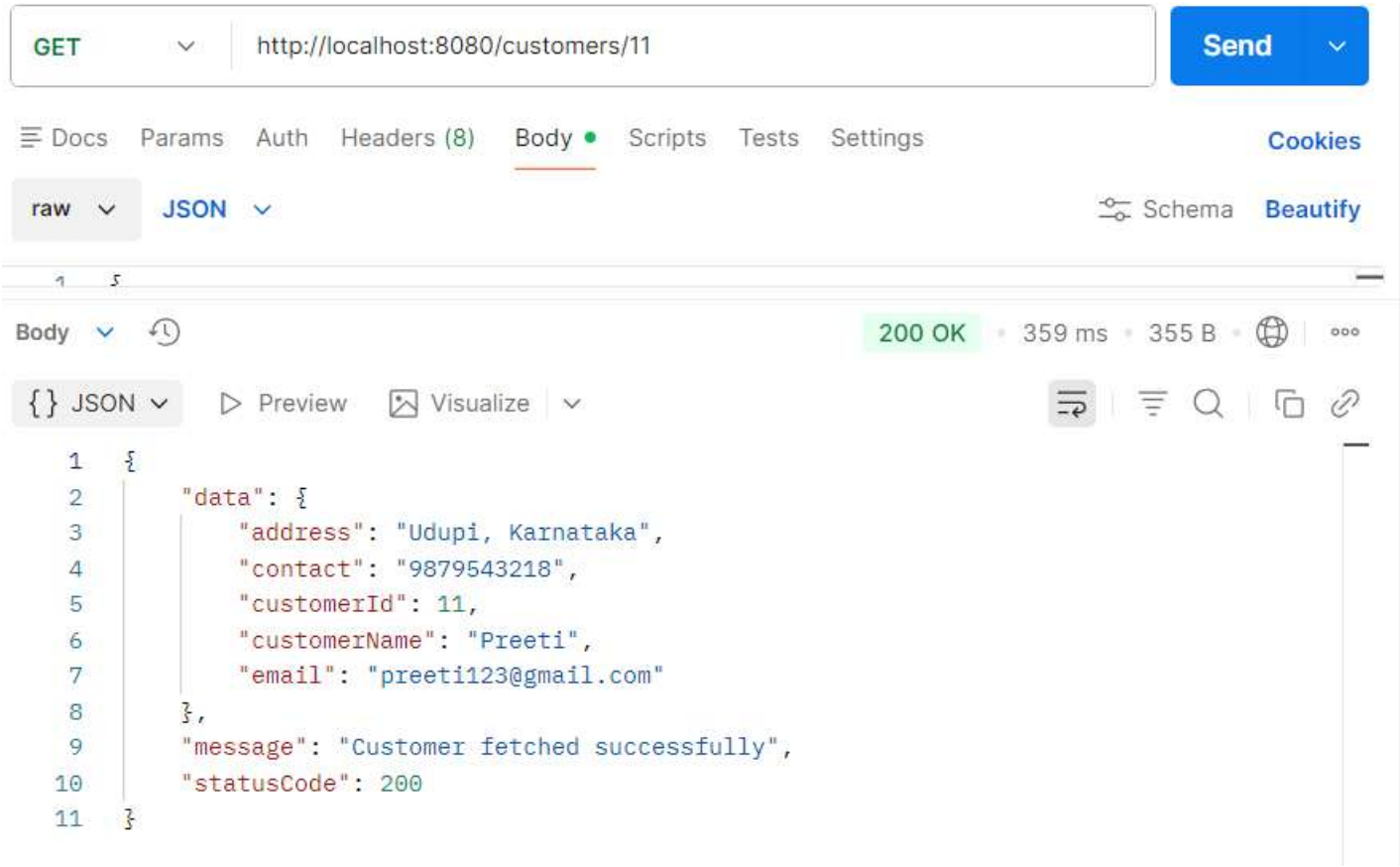
3. Get Customer By Id**(CustomerService.java)**

```java
// 3. Get Customer By Id
public ResponseEntity<ResponseStructure<Customer>> getCustomerById(Integer id) {
    Optional<Customer> opt = repository.findById(id);
    if(opt.isEmpty()) {
        throw new ResourceNotFoundException("Customer not fuond with id: " + id);
    }
    ResponseStructure<Customer> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Customer fetched successfully");
    response.setData(opt.get());
    return new ResponseEntity<ResponseStructure<Customer>>(response, HttpStatus.OK);
}
```

CustomerController.java

```java
// 3. Get Customer By Id
@GetMapping("/{id}")
public ResponseEntity<ResponseStructure<Customer>> getCustomerById(@PathVariable Integer id) {
    return service.getCustomerById(id);
}
```

Output(Postman) :



4. Update Customer(**CustomerService.java)**

```java
// 4. Update Customer
public ResponseEntity<ResponseStructure<Customer>> updateCustomer(Integer id, Customer customer) {
    Optional<Customer> opt = repository.findById(id);
    ResponseStructure<Customer> response = new ResponseStructure<>();
    if(opt.isPresent()) {
        customer.setCustomerId(id);
        Customer updatedCustomer = repository.save(customer);
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Customer updated successfully");
        response.setData(updatedCustomer);
        return new ResponseEntity<ResponseStructure<Customer>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("Customer not found with id: " + id);
    }
}
```

CustomerController.java

```java
// 4. Update Customer
@PutMapping("/{id}")
public ResponseEntity<ResponseStructure<Customer>> updateCustomer(@PathVariable Integer id, @RequestBody Customer customer) {
    return service.updateCustomer(id, customer);
}
```

Output(Postman) :

PUT http://localhost:8080/customers/11 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1  {
2    "customerName": "Preeti Sahu",
3    "email": "preeti@gmail.com",
4    "contact": "9879363218",
5    "address": "Udupi, Karnataka"
6  }
```

Body 200 OK 84 ms 357 B

{} JSON Preview Visualize

```
1  {
2      "data": {
3          "address": "Udupi, Karnataka",
4          "contact": "9879363218",
5          "customerId": 11,
6          "customerName": "Preeti Sahu",
7          "email": "preeti@gmail.com"
8      },
9      "message": "Customer updated successfully",
10     "statusCode": 200
11 }
```

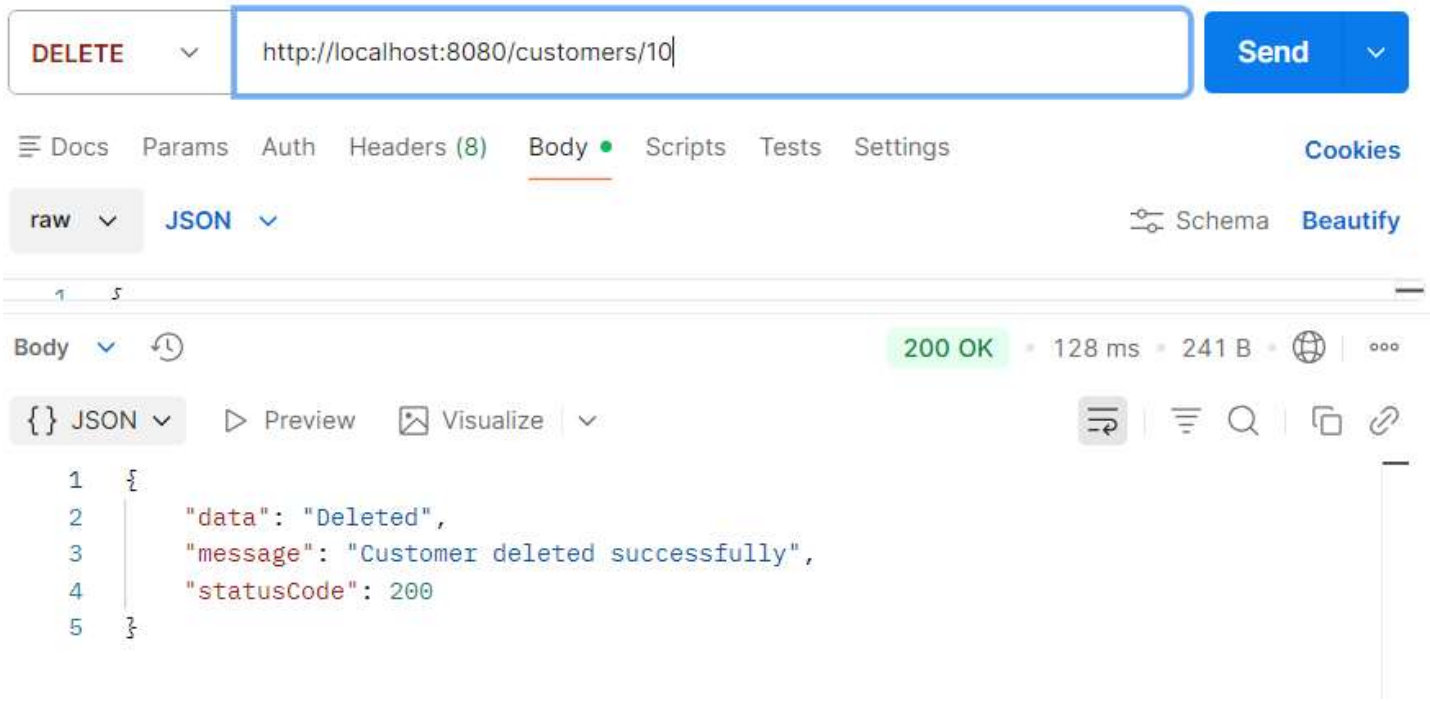
5. Delete Customer(**CustomerService.java)**

```
// 5. delete customer
public ResponseEntity<ResponseStructure<String>> deleteCustomer(Integer id) {
    Optional<Customer> opt = repository.findById(id);
    ResponseStructure<String> response = new ResponseStructure<>();
    if(opt.isPresent()) {
        repository.deleteById(id);
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Customer deleted successfully");
        response.setData("Deleted");
        return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.OK);
    }
    throw new ResourceNotFoundException("Customer not found with id: " + id);
}
```

CustomerController.java

```
// 5. Delete Customer
@DeleteMapping("/{id}")
public ResponseEntity<ResponseStructure<String>> deleteCustomer(@PathVariable Integer id) {
    return service.deleteCustomer(id);
}
```

Output(Postman) :



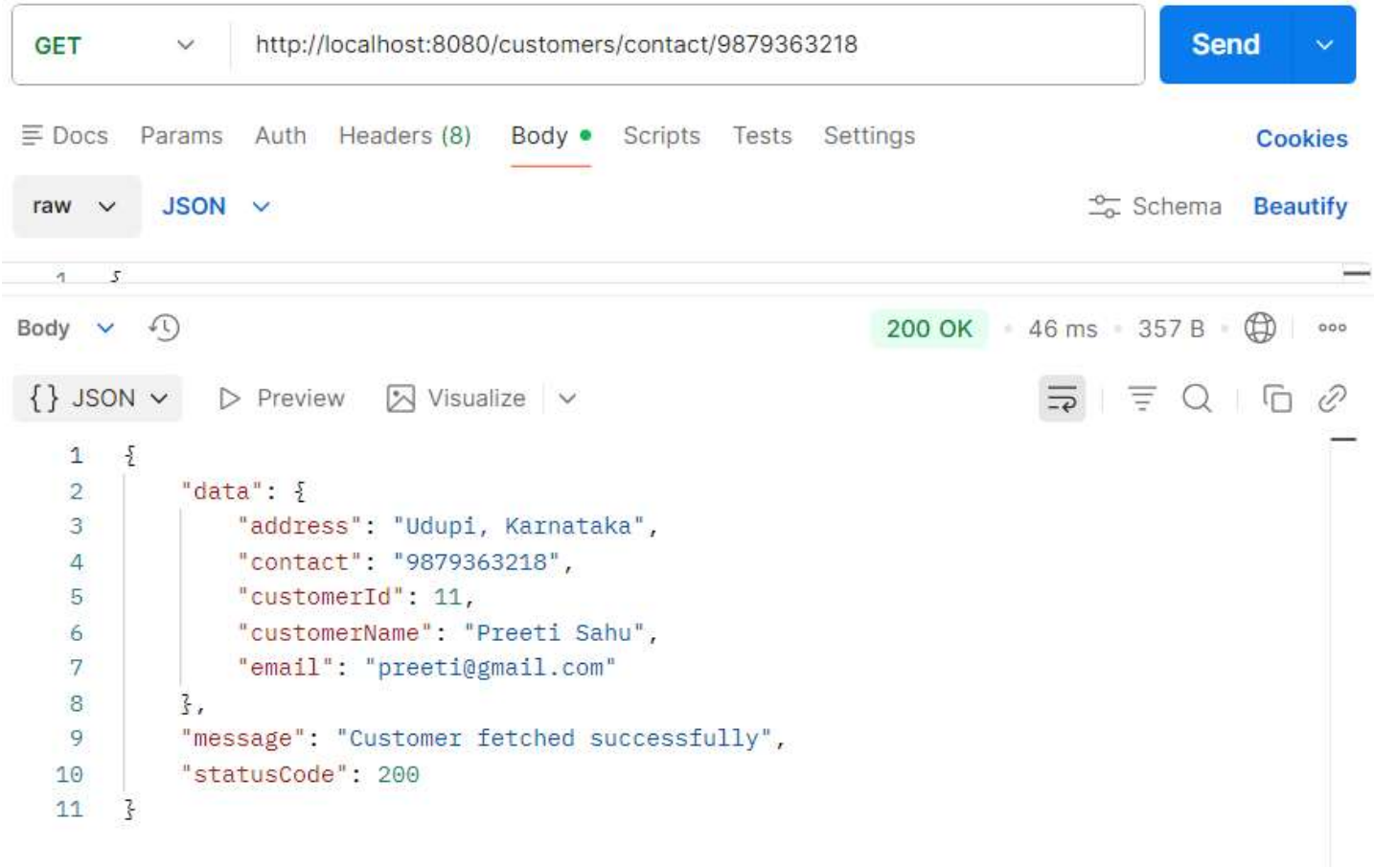
6. get by contact(**CustomerService.java)**

```java
// 6. get by contact
public ResponseEntity<ResponseStructure<Customer>> getCustomerByContact(String contact) {
    Optional<Customer> opt = repository.findByContact(contact);
    if (opt.isEmpty()) {
        throw new ResourceNotFoundException("Customer not found with contact: " + contact);
    }
    ResponseStructure<Customer> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Customer fetched successfully");
    response.setData(opt.get());
    return new ResponseEntity<ResponseStructure<Customer>>(response, HttpStatus.OK);
}
```

CustomerController.java

```java
// 6. Get Customer By Contact
@GetMapping("/contact/{contact}")
public ResponseEntity<ResponseStructure<Customer>> getCustomerByContact(@PathVariable String contact) {
    return service.getCustomerByContact(contact);
}
```

Output(Postman) :



8.3 Restaurant

Restaurant.java

```java
package jsp.springboot.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
@Entity
public class Restaurant {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer restaurantId;
    private String restaurantName;
    private String location;

    @OneToMany(mappedBy = "restaurant")
    @JsonIgnore
    private List<Item> items;

    // getter & setter


}
```

RestaurantRepository.java

```java
package jsp.springboot.repository;

import java.util.List;
import org.springframework.data.domain.Page;
import org.springframework.data.jpa.repository.JpaRepository;
import jsp.springboot.entity.Restaurant;

public interface RestaurantRepository extends JpaRepository<Restaurant, Integer> {

    // 6. Get Restaurant by Location
    List<Restaurant> findByLocation(String location);

    // 7. Get Restaurant by Name
    List<Restaurant> findByRestaurantName(String name);

}
```

RestaurantService.java

```java
package jsp.springboot.service;

import java.util.List;
import java.util.Optional;
import org.springframework.data.domain.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Item;
import jsp.springboot.entity.Restaurant;
import jsp.springboot.exception.ResourceNotFoundException;
import jsp.springboot.repository.RestaurantRepository;

@Service
public class RestaurantService {
    @Autowired
    private RestaurantRepository repository;

    // 1. Add Restaurant
    public ResponseEntity<ResponseStructure<Restaurant>> addRestaurant(Restaurant restaurant) {

        if(restaurant.getRestaurantName() == null || restaurant.getRestaurantName().isEmpty())
            throw new ResourceNotFoundException("Restaurant name is required");

        if(restaurant.getLocation() == null || restaurant.getLocation().isEmpty())
            throw new ResourceNotFoundException("Restaurant location is required");

        ResponseStructure<Restaurant> response = new ResponseStructure<>();

        response.setStatusCode(HttpStatus.CREATED.value());
        response.setMessage("Restaurant created successfully");
        response.setData(repository.save(restaurant));

        return new ResponseEntity<ResponseStructure<Restaurant>>(response, HttpStatus.CREATED);
    }
```

RestaurantController.java

```java
package jsp.springboot.controller;

import org.springframework.data.domain.*;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Item;
import jsp.springboot.entity.Restaurant;
import jsp.springboot.service.RestaurantService;

@RestController
@RequestMapping("/restaurants")
public class RestaurantController {

    @Autowired
    private RestaurantService service;

    // 1. Add Restaurant
    @PostMapping
    public ResponseEntity<ResponseStructure<Restaurant>> addRestaurant(@RequestBody Restaurant restaurant) {
        return service.addRestaurant(restaurant);
    }
```

Output(Postman) :

POST http://localhost:8080/restaurants Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "restaurantName": "Royal Kitchen",
3   "location": "Delhi"
4 }
```

Body 201 Created 40 ms 309 B

{} JSON Preview Visualize

```
1 {
2     "data": {
3         "location": "Delhi",
4         "restaurantId": 5,
5         "restaurantName": "Royal Kitchen"
6     },
7     "message": "Restaurant created successfully",
8     "statusCode": 201
9 }
```

pgAdmin :

public.restaurant/food_order_db/postgres@PostgreSQL 17

No limit

Query Query History

```
1 SELECT * FROM public.restaurant
2 ORDER BY restaurant_id ASC
```

Data Output Messages Notifications

	restaurant_id [PK] integer	location character varying (255)	restaurant_name character varying (255)
1	1	Bangalore	Spicy Delight
2	2	Chennai	Food Palace
3	3	Hyderabad	Taste Hub
4	4	Mumbai	Urban Bites
5	5	Delhi	Royal Kitchen

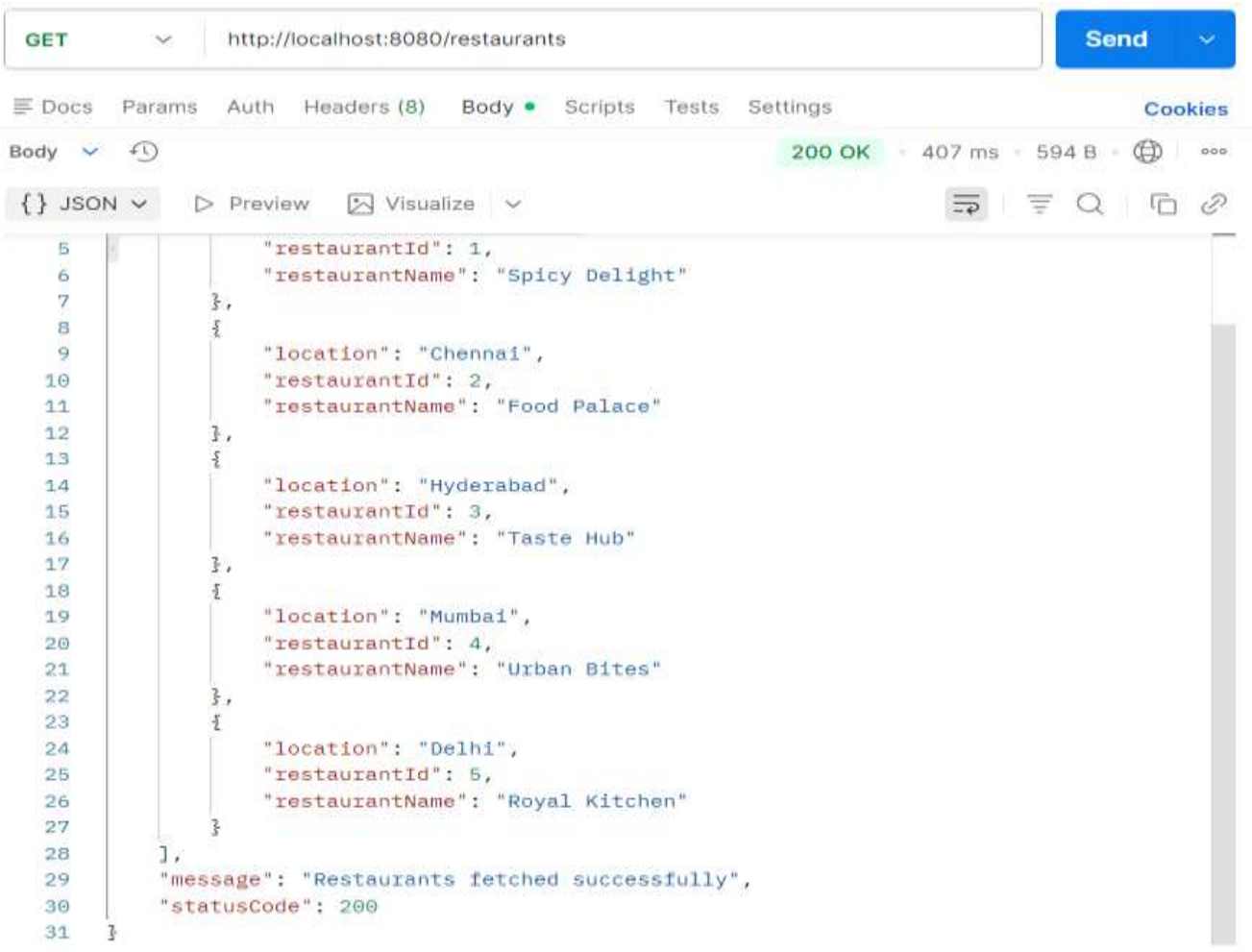
2. Get All Restaurants API(RestaurantService.java)

```java
// 2. Get All Restaurants API
public ResponseEntity<ResponseStructure<List<Restaurant>>> getAllRestaurants() {
    List<Restaurant> restaurants = repository.findAll();
    ResponseStructure<List<Restaurant>> response = new ResponseStructure<List<Restaurant>>();
    if(!restaurants.isEmpty()) {
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Restaurants fetched successfully");
        response.setData(restaurants);
        return new ResponseEntity<ResponseStructure<List<Restaurant>>>(response, HttpStatus.OK);
    }
    else {
        throw new ResourceNotFoundException("No restaurants found");
    }
}
```

RestaurantController.java

```java
// 2. Get All Restaurants API
@GetMapping
public ResponseEntity<ResponseStructure<List<Restaurant>>> getAllRestaurants() {
    return service.getAllRestaurants();
}
```

Output(Postman) :



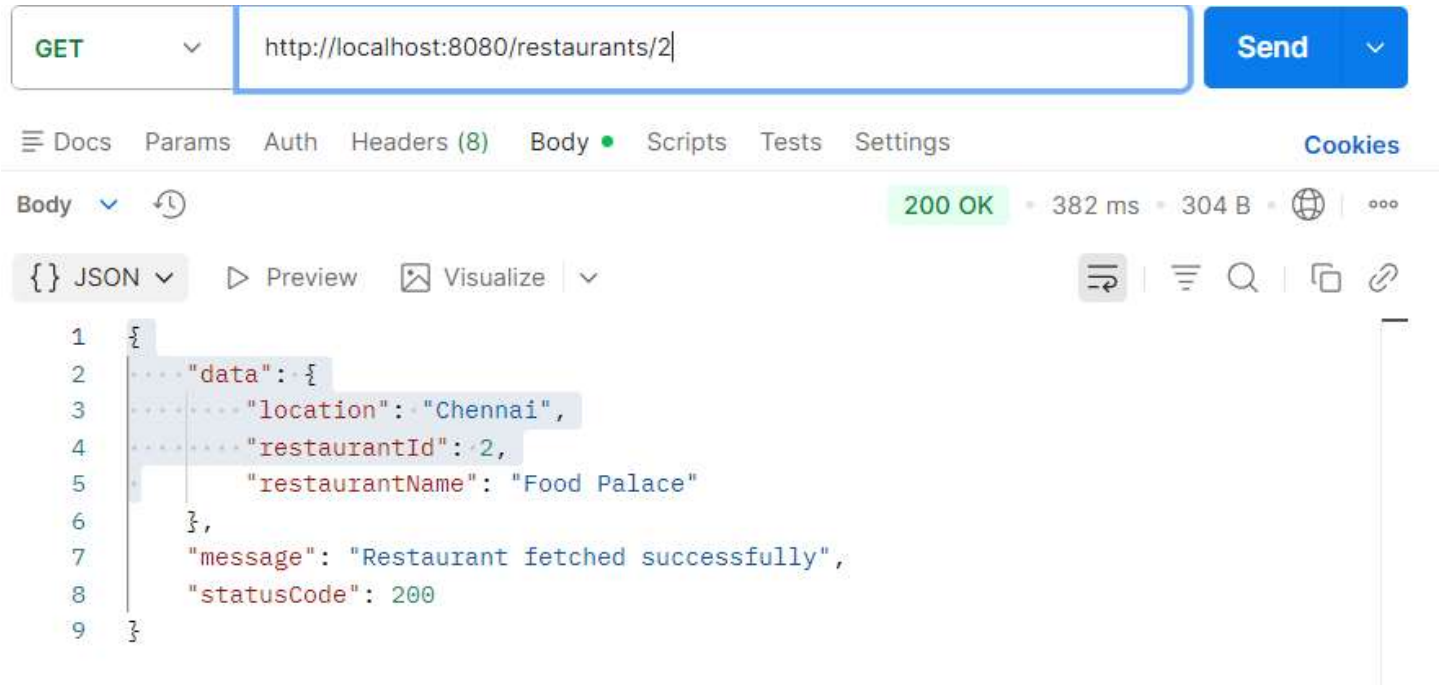
3. get Restaurant by id(RestaurantService.java)

```java
// 3. get Restaurant by id
public ResponseEntity<ResponseStructure<Restaurant>> getRestaurantById(Integer id) {
    Optional<Restaurant> opt = repository.findById(id);
    ResponseStructure<Restaurant> response = new ResponseStructure<>();
    if (opt.isPresent()) {
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Restaurant fetched successfully");
    response.setData(opt.get());
    return new ResponseEntity<ResponseStructure<Restaurant>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("Restaurant not found with id: " + id);
    }
}
```

RestaurantController.java

```java
// 3. get Restaurant by id
@GetMapping("/{id}")
public ResponseEntity<ResponseStructure<Restaurant>> getRestaurantById(@PathVariable Integer id) {
    return service.getRestaurantById(id);
}
```

Output(Postman) :



4. Update Restaurant(RestaurantService.java)

```java
// 4. Update Restaurant
public ResponseEntity<ResponseStructure<Restaurant>> updateRestaurant(Integer id, Restaurant restaurant) {
    Optional<Restaurant> opt = repository.findById(id);
    ResponseStructure<Restaurant> response = new ResponseStructure<>();
    if (opt.isPresent()) {
        restaurant.setRestaurantId(id);
        Restaurant updatedRestaurant = repository.save(restaurant);
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Restaurant updated successfully");
        response.setData(updatedRestaurant);
        return new ResponseEntity<ResponseStructure<Restaurant>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("Restaurant not found with id: " + id);
    }
}
```

RestaurantController.java

```java
// 4. update Restaurant
@PutMapping("/{id}")
public ResponseEntity<ResponseStructure<Restaurant>> updateRestaurant(@PathVariable Integer id, @RequestBody Restaurant restaurant) {
    return service.updateRestaurant(id, restaurant);
}
```

Output(Postman) :

PUT http://localhost:8080/restaurants/5 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "restaurantName": "Royal Kitchen",
3   "location": "Mumbai"
4 }
```

Body 200 OK 363 ms 305 B

{} JSON Preview Visualize

```
1 {
2     "data": {
3         "location": "Mumbai",
4         "restaurantId": 5,
5         "restaurantName": "Royal Kitchen"
6     },
7     "message": "Restaurant updated successfully",
8     "statusCode": 200
9 }
```

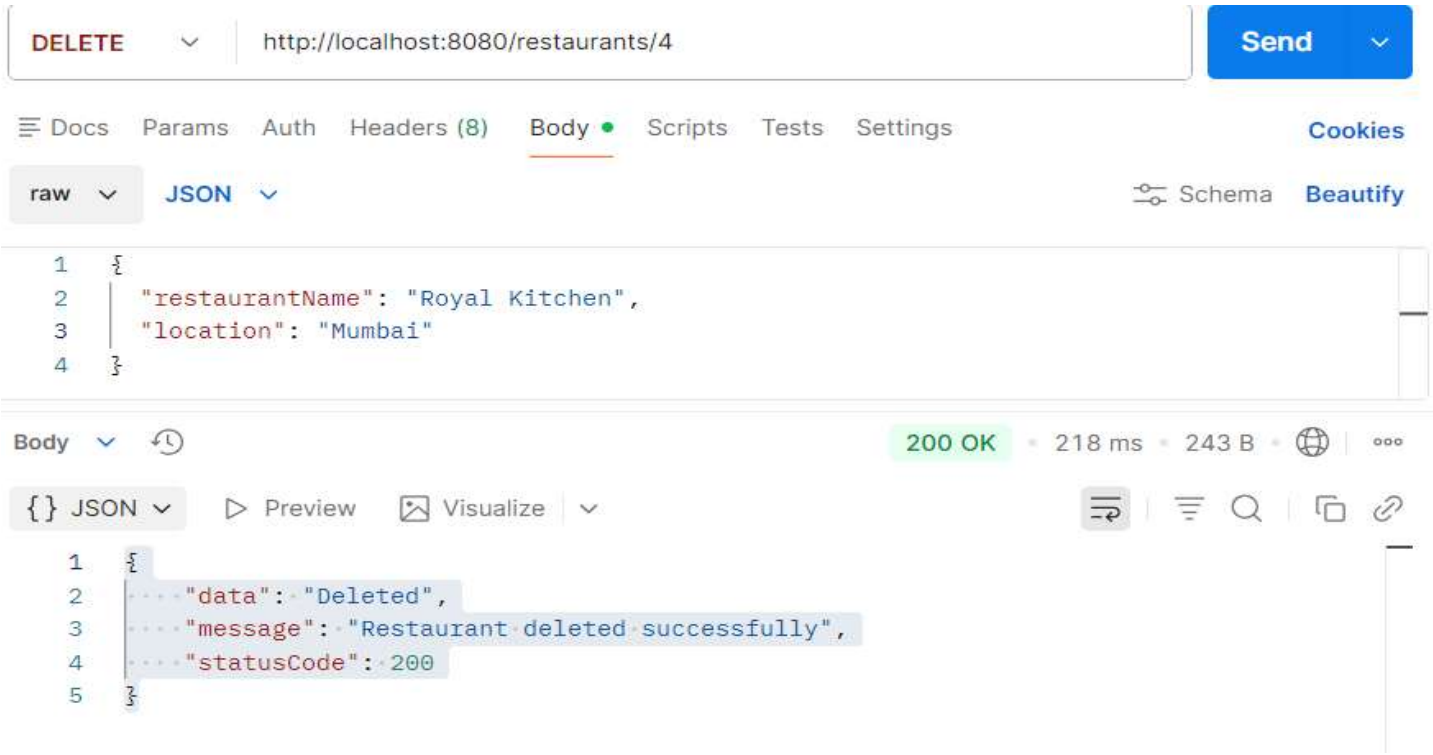
5. Delete Restaurant(RestaurantService.java)

```java
// 5. Delete Restaurant
public ResponseEntity<ResponseStructure<String>> deleteRestaurant(Integer id) {
    Optional<Restaurant> opt = repository.findById(id);
    ResponseStructure<String> response = new ResponseStructure<>();
    if(opt.isPresent()) {
        repository.deleteById(id);
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Restaurant deleted successfully");
        response.setData("Deleted");
        return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("Restaurant not found with id: " + id);
    }
}
```

RestaurantController.java

```java
// 5. Delete Restaurant
@DeleteMapping("/{id}")
public ResponseEntity<ResponseStructure<String>> deleteRestaurant(@PathVariable Integer id) {
    return service.deleteRestaurant(id);
}
```

Output(Postman) :



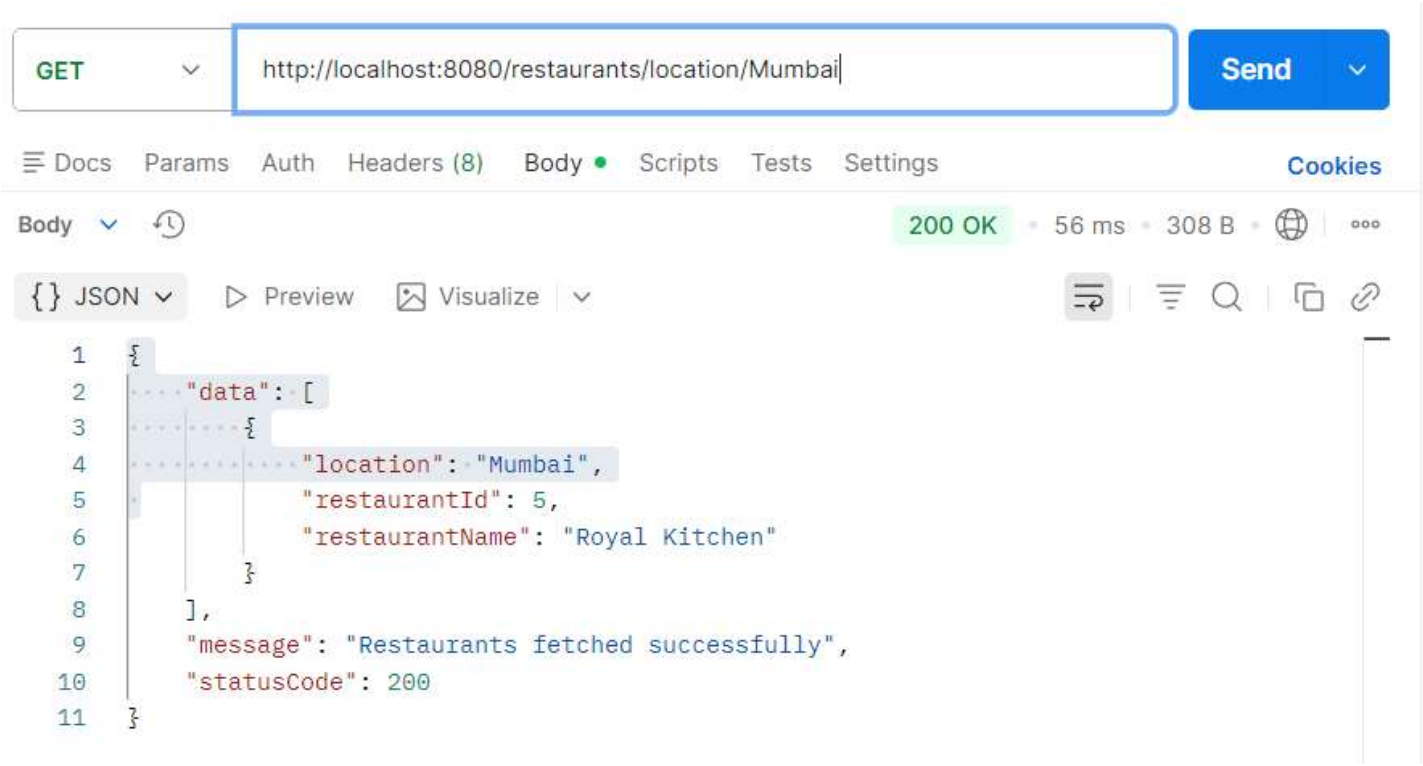
6. get Restaurant by location(RestaurantService.java)

```java
// 6. Get Restaurant by Location
public ResponseEntity<ResponseStructure<List<Restaurant>>> getRestaurantByLocation(String location) {
    List<Restaurant> restaurants = repository.findByLocation(location);
    if (restaurants.isEmpty()) {
        throw new ResourceNotFoundException("No restaurants found in location: " + location);
    }
    ResponseStructure<List<Restaurant>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Restaurants fetched successfully");
    response.setData(restaurants);
    return new ResponseEntity<ResponseStructure<List<Restaurant>>>(response, HttpStatus.OK);
}
```

RestaurantController.java

```java
// 6. get Restaurant by location
@GetMapping("/location/{location}")
public ResponseEntity<ResponseStructure<List<Restaurant>>> getRestaurantByLocation(@PathVariable String location) {
    return service.getRestaurantByLocation(location);
}
```

Output(Postman) :



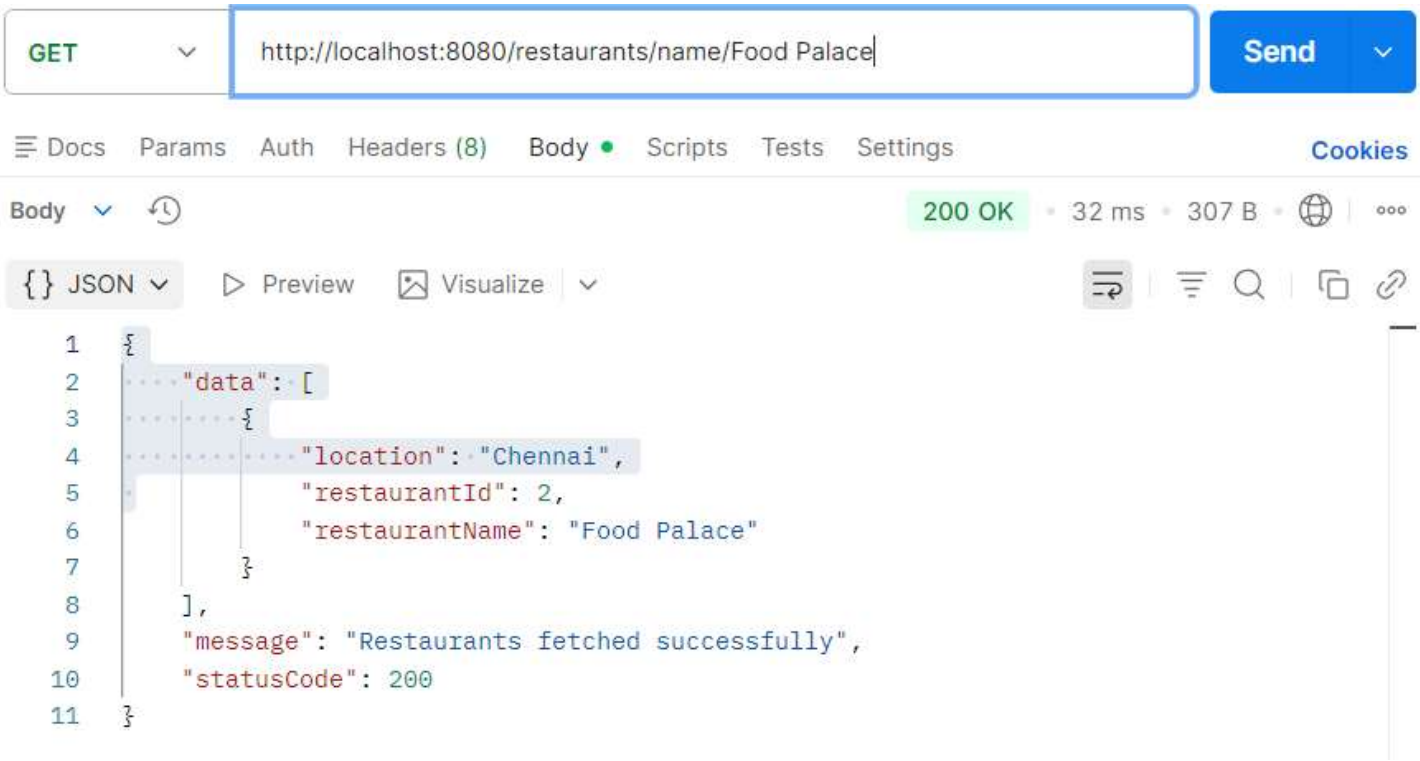
7. Get Restaurant by Name(RestaurantService.java)

```java
// 7. Get Restaurant by Name
public ResponseEntity<ResponseStructure<List<Restaurant>>> getRestaurantByName(String name) {
    List<Restaurant> restaurants = repository.findByRestaurantName(name);
    if (restaurants.isEmpty()) {
        throw new ResourceNotFoundException("No restaurants found with name: " + name);
    }
    ResponseStructure<List<Restaurant>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Restaurants fetched successfully");
    response.setData(restaurants);
    return new ResponseEntity<ResponseStructure<List<Restaurant>>>(response, HttpStatus.OK);
}
```

RestaurantController.java

```java
// 7. Get Restaurant by Name
@GetMapping("/name/{name}")
public ResponseEntity<ResponseStructure<List<Restaurant>>> getRestaurantByName(@PathVariable String name) {
    return service.getRestaurantByName(name);
}
```

Output(Postman) :



8. Get Menu Items of Restaurant(RestaurantService.java)

```java
// 8. Get Menu Items of Restaurant
public ResponseEntity<ResponseStructure<List<Item>>> getMenuItemsOfRestaurant(Integer restaurantId) {
    Optional<Restaurant> opt = repository.findById(restaurantId);
    if (opt.isEmpty()) {
        throw new ResourceNotFoundException("Restaurant not found with id: " + restaurantId);
    }
    Restaurant restaurant = opt.get();
    List<Item> items = restaurant.getItems();

    if (items == null || items.isEmpty()) {
        throw new ResourceNotFoundException("No menu items found for restaurant id: " + restaurantId);
    }

    ResponseStructure<List<Item>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Menu items fetched successfully");
    response.setData(items);
    return new ResponseEntity<ResponseStructure<List<Item>>>(response, HttpStatus.OK);
}
```

RestaurantController.java

```java
// 8. Get Menu Items of Restaurant
@GetMapping("/{id}/items")
public ResponseEntity<ResponseStructure<List<Item>>> getMenuItemsOfRestaurant(@PathVariable Integer id) {
    return service.getMenuItemsOfRestaurant(id);
}
```

Output(Postman) :

GET http://localhost:8080/restaurants/1/items Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 1.07 s 546 B

{} JSON Preview Visualize


```
{
    "data": [
        {
            "available": true,
            "itemId": 1,
            "itemName": "Veg Burger",
            "price": 120.0,
            "restaurant": {
                "location": "Bangalore",
                "restaurantId": 1,
                "restaurantName": "Spicy Delight"
            }
        },
        {
            "available": true,
            "itemId": 6,
            "itemName": "Pav Bhaji",
            "price": 140.0,
            "restaurant": {
                "location": "Bangalore",
                "restaurantId": 1,
                "restaurantName": "Spicy Delight"
            }
        }
    ],
    "message": "Menu items fetched successfully",
    "statusCode": 200
```

11.Get Restaurants with Pagination and Sorting(RestaurantService.java)

```java
// 9. Get Restaurants with Pagination and Sorting
public ResponseEntity<ResponseStructure<Page<Restaurant>>>
getRestaurantsByPaginationAndSorting(int pageNumber, int pageSize, String field) {
    Pageable pageable = PageRequest.of(pageNumber, pageSize, Sort.by(field));
    Page<Restaurant> restaurants = repository.findAll(pageable);
    if (restaurants.isEmpty()) {
        throw new ResourceNotFoundException("No restaurants found");
    }

    ResponseStructure<Page<Restaurant>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Restaurants fetched successfully");
    response.setData(restaurants);

    return new ResponseEntity<ResponseStructure<Page<Restaurant>>>(response, HttpStatus.OK);
}
```

RestaurantController.java

```java
// 9. Get Restaurants with Pagination and Sorting
@GetMapping("/page/{pageNumber}/{pageSize}/{field}")
public ResponseEntity<ResponseStructure<Page<Restaurant>>> getRestaurantsByPaginationAndSorting(@PathVariable int pageNumber, @PathVariable int pageSize, @PathVariable String field) {
    return service.getRestaurantsByPaginationAndSorting(pageNumber, pageSize, field);
}
```

Output(Postman) :

GET http://localhost:8080/restaurants/page/1/3/location Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 135 ms 623 B

{} JSON Preview Visualize


```
12          "last": true,
13          "number": 1,
14          "numberOfElements": 1,
15          "pageable": {
16              "offset": 3,
17              "pageNumber": 1,
18              "pageSize": 3,
19              "paged": true,
20              "sort": {
21                  "empty": false,
22                  "sorted": true,
23                  "unsorted": false
24              },
25              "unpaged": false
26          },
27          "size": 3,
28          "sort": {
29              "empty": false,
30              "sorted": true,
31              "unsorted": false
32          },
33          "totalElements": 4,
34          "totalPages": 2
35      },
36      "message": "Restaurants fetched successfully",
37      "statusCode": 200
38  }
```

8.4. Menu Item :

Item.java

```java
package jsp.springboot.entity;

import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;

@Entity
public class Item {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer itemId;
    private String itemName;
    private Double price;
    @Column(nullable = false)
    private boolean available = true;

    @ManyToOne
    @JoinColumn(name = "restaurant_id")
    private Restaurant restaurant;

    @OneToMany(mappedBy = "item")
    @JsonIgnore
    private List<OrderItem> orderItems;

    // getter and setter


}
```

MenuItemRepository.java

```java
package jsp.springboot.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import jsp.springboot.entity.Item;

public interface MenuItemRepository extends JpaRepository<Item, Integer> {
    // 6. Get Items Greater than price
    List<Item> findByPriceGreaterThan(Double price);

    // 7. Get Item by name
    List<Item> findByItemName(String itemName);
}
```

MenuItemService.java

```java
package jsp.springboot.service;

import java.util.List;

@Service
public class MenuItemService {

    @Autowired
    private MenuItemRepository itemRepository;

    @Autowired
    private RestaurantRepository restaurantRepository;

    // 1. Add Menu Item
    public ResponseEntity<ResponseStructure<Item>> addMenuItem(Item item) {

        if(item.getPrice() < 0)
            throw new InvalidInputException("Price cannot be negative");

        if(item.getRestaurant() == null)
            throw new InvalidInputException("Restaurant required");

        Restaurant restaurant = restaurantRepository
                .findById(item.getRestaurant().getRestaurantId())
                .orElseThrow(() -> new ResourceNotFoundException("Restaurant not found"));

        item.setRestaurant(restaurant);

        ResponseStructure<Item> response = new ResponseStructure<>();
        response.setStatusCode(HttpStatus.CREATED.value());
        response.setMessage("Menu item added successfully");
        response.setData(itemRepository.save(item));

        return new ResponseEntity<ResponseStructure<Item>>(response, HttpStatus.CREATED);
    }
```

```java
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Item;
import jsp.springboot.entity.Restaurant;
import jsp.springboot.exception.InvalidInputException;
import jsp.springboot.exception.ResourceNotFoundException;
import jsp.springboot.repository.MenuItemRepository;
import jsp.springboot.repository.RestaurantRepository;
```

MenuItemController.java

```java
package jsp.springboot.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Item;
import jsp.springboot.service.MenuItemService;

@RestController
@RequestMapping("/menuItem")
public class MenuItemController {

    @Autowired
    private MenuItemService service;

    // 1. Add Menu Item
    @PostMapping
    public ResponseEntity<ResponseStructure<Item>> addMenuItem(@RequestBody Item item) {
        return service.addMenuItem(item);
    }
```

Output(Postman) :

POST http://localhost:8080/menuItem Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
{
  "itemName": "Veg Burger",
  "price": 120,
  "available": true,
  "restaurant": {
    "restaurantId": 1
  }
}
```

Body 201 Created 171 ms 391 B

{} JSON Preview Visualize

```
{
    "data": {
        "available": true,
        "itemId": 1,
        "itemName": "Veg Burger",
        "price": 120.0,
        "restaurant": {
            "location": "Bangalore",
            "restaurantId": 1,
            "restaurantName": "Spicy Delight"
        }
    },
    "message": "Menu item added successfully",
    "statusCode": 201
}
```

pgAdmin :

Show

	item_id [PK] integer	available boolean	item_name character varying (255)	price double precision	restaurant_id integer
1	1	true	Veg Burger	120	1
2	2	true	Paneer Pizza	250	2
3	3	true	Masala Dosa	90	3
4	4	true	Veg Fried Rice	180	5
5	5	true	Pav Bhaji	140	1
6	6	true	Chole Bhature	160	2
7	7	true	Veg Noodles	170	3
8	8	true	Paneer Butter Masala	220	5
9	9	true	Idli Sambar	80	1
10	10	true	Veg Sandwich	110	2

2. Get All Menu Item(MenuItemService.java) :

```java
// 2. Get All Menu Item
public ResponseEntity<ResponseStructure<List<Item>>> getAllMenuItems() {
    List<Item> items = itemRepository.findAll();
    if(!items.isEmpty()) {
        ResponseStructure<List<Item>> response = new ResponseStructure<>();
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Menu items fetched successfully");
        response.setData(items);
        return new ResponseEntity<ResponseStructure<List<Item>>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("No menu items found");
    }
}
```

MenuItemController.java

```java
// 2. Get All Menu Item
@GetMapping
public ResponseEntity<ResponseStructure<List<Item>>> getAllMenuItems(){
    return service.getAllMenuItems();
}
```

Output(Postman) :

GET http://localhost:8080/menuItem Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 333 ms 1.75 KB

{} JSON Preview Visualize


```
90          },
91          {
92              "available": true,
93              "itemId": 9,
94              "itemName": "Idli Sambar",
95              "price": 80.0,
96              "restaurant": {
97                  "location": "Bangalore",
98                  "restaurantId": 1,
99                  "restaurantName": "Spicy Delight"
100             }
101         },
102         {
103             "available": true,
104             "itemId": 10,
105             "itemName": "Veg Sandwich",
106             "price": 110.0,
107             "restaurant": {
108                 "location": "Chennai",
109                 "restaurantId": 2,
110                 "restaurantName": "Food Palace"
111             }
112         }
113     ],
114     "message": "Menu items fetched successfully",
115     "statusCode": 200
116 }
```

3. Get Menu Item By ID(MenuItemService.java) :

```java
// 3. Get Menu Item By ID
public ResponseEntity<ResponseStructure<Item>> getMenuItemById(Integer id){
    Optional<Item> opt = itemRepository.findById(id);
    ResponseStructure<Item> response = new ResponseStructure<>();
    if(opt.isPresent()) {
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Menu item fetched successfully");
        response.setData(opt.get());
        return new ResponseEntity<ResponseStructure<Item>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("Menu item not found");
    }
}
```

MenuItemController.java

```java
// 3. Get Menu Item By ID
@GetMapping("/{id}")
public ResponseEntity<ResponseStructure<Item>> getMenuItemById(@PathVariable Integer id){
    return service.getMenuItemById(id);
}
```

Output(Postman) :

GET http://localhost:8080/menuItem/1 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 100 ms 388 B

{} JSON Preview Visualize


```
1  {
2      "data": {
3          "available": true,
4          "itemId": 1,
5          "itemName": "Veg Burger",
6          "price": 120.0,
7          "restaurant": {
8              "location": "Bangalore",
9              "restaurantId": 1,
10             "restaurantName": "Spicy Delight"
11         }
12     },
13     "message": "Menu item fetched successfully",
14     "statusCode": 200
15 }
```

4. Update Price(MenuItemService.java) :

```java
// 4. Update Price
public ResponseEntity<ResponseStructure<Item>> updatePrice(Integer id, Double price) {
    if(price <= 0)
        throw new InvalidInputException("Price cannot be negative or Zero");
    Item item = itemRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Menu item not found"));
    item.setPrice(price);
    ResponseStructure<Item> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Price updated successfully");
    response.setData(itemRepository.save(item));
    return new ResponseEntity<ResponseStructure<Item>>(response, HttpStatus.OK);
}
```

MenuItemController.java

```java
// 4. Update Price
// http://localhost:8080/menuItem/3/price?price=200
@PutMapping("/{id}/price")
public ResponseEntity<ResponseStructure<Item>> updatePrice(@PathVariable Integer id, @RequestParam Double price){
    return service.updatePrice(id, price);
}
```

Output(Postman) :

PUT http://localhost:8080/menuItem/3/price?price=200 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 163 ms 381 B

{} JSON Preview Visualize


```
{
    "data": {
        "available": true,
        "itemId": 3,
        "itemName": "Masala Dosa",
        "price": 200.0,
        "restaurant": {
            "location": "Hyderabad",
            "restaurantId": 3,
            "restaurantName": "Taste Hub"
        }
    },
    "message": "Price updated successfully",
    "statusCode": 200
}
```

5. Delete Menu Item(MenuItemService.java) :

```java
// 5. Delete Menu Item
public ResponseEntity<ResponseStructure<String>> deleteMenuItem(Integer id) {
    Item item = itemRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Menu item not found"));
    itemRepository.delete(item);
    ResponseStructure<String> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Menu item deleted successfully");
    response.setData("Deleted");
    return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.OK);
}
```

MenuItemController.java

```java
// 5. Delete Menu Item
@DeleteMapping("/{id}")
public ResponseEntity<ResponseStructure<String>> deleteMenuItem(@PathVariable Integer id){
    return service.deleteMenuItem(id);
}
```

Output(Postman) :

6. Get Items Greater Than Price(MenuItemService.java) :

```java
// 6. Get Items Greater Than Price
public ResponseEntity<ResponseStructure<List<Item>>> getItemsGreaterThanPrice(Double price) {
    List<Item> items = itemRepository.findByPriceGreaterThan(price);
    ResponseStructure<List<Item>> response = new ResponseStructure<>();
    if(!items.isEmpty()) {
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Items fetched successfully");
        response.setData(items);
        return new ResponseEntity<ResponseStructure<List<Item>>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("No items found greater than given price");
    }
}
```

MenuItemController.java

```java
// 6. Get Items Greater Than Price
// http://localhost:8080/menuItem/price?price=200
@GetMapping("/price")
public ResponseEntity<ResponseStructure<List<Item>>> getItemsGreaterThanPrice(@RequestParam Double price){
    return service.getItemsGreaterThanPrice(price);
}
```

Output(Postman) :

GET http://localhost:8080/menuItem/price?price=200 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 117 ms 547 B

{} JSON Preview Visualize


```
2     "data": [
3         {
4             "available": true,
5             "itemId": 2,
6             "itemName": "Paneer Pizza",
7             "price": 250.0,
8             "restaurant": {
9                 "location": "Chennai",
10                "restaurantId": 2,
11                "restaurantName": "Food Palace"
12            }
13        },
14        {
15            "available": true,
16            "itemId": 8,
17            "itemName": "Paneer Butter Masala",
18            "price": 220.0,
19            "restaurant": {
20                "location": "Mumbai",
21                "restaurantId": 5,
22                "restaurantName": "Royal Kitchen"
23            }
24        }
25    ],
26    "message": "Items fetched successfully",
27    "statusCode": 200
28 }
```

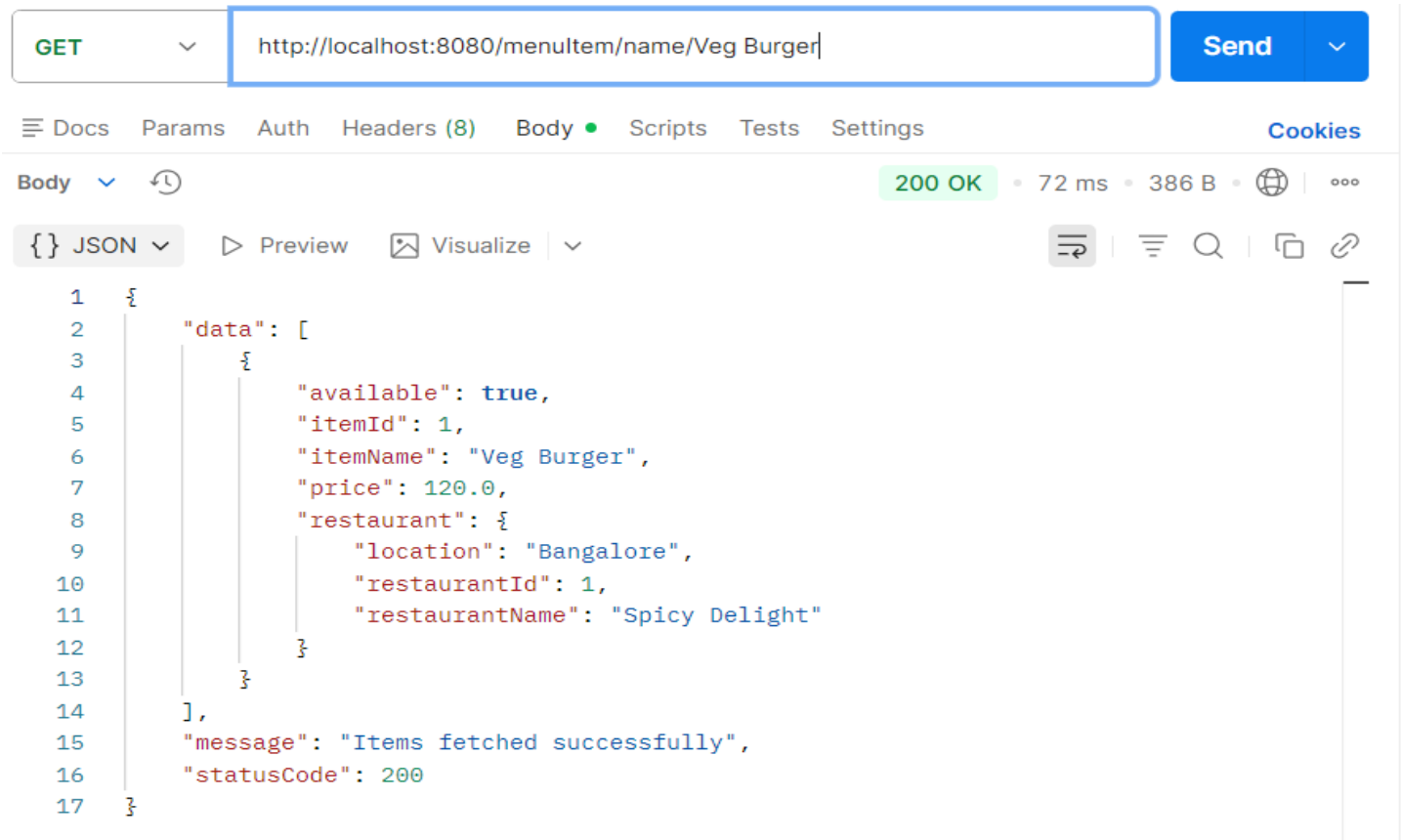
7. Get Item By Name(MenuItemService.java) :

```java
// 7. Get Item By Name
public ResponseEntity<ResponseStructure<List<Item>>> getItemByName(String name) {
    List<Item> items = itemRepository.findByItemName(name);
    ResponseStructure<List<Item>> response = new ResponseStructure<>();
    if(!items.isEmpty()) {
        response.setStatusCode(HttpStatus.OK.value());
        response.setMessage("Items fetched successfully");
        response.setData(items);
        return new ResponseEntity<ResponseStructure<List<Item>>>(response, HttpStatus.OK);
    } else {
        throw new ResourceNotFoundException("Item not found");
    }
}
```

MenuItemController.java

```java
// 7. Get Item By Name
@GetMapping("/name/{name}")
public ResponseEntity<ResponseStructure<List<Item>>> getItemByName(@PathVariable String name){
    return service.getItemByName(name);
}
```

Output(Postman) :



8.5. Order :

Order.java

```java
package jsp.springboot.entity;

import java.time.LocalDateTime;
import java.util.List;
import jakarta.persistence.*;
import jsp.springboot.enums.OrderStatus;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer orderId;

    private LocalDateTime orderDateTime;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    private Double totalAmount;

    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
    private List<OrderItem> orderItems;

    @OneToOne(mappedBy = "order", cascade = CascadeType.ALL)
    private Payment payment;

    // getter and setter
}
```

OrderRepository.java

```java
package jsp.springboot.repository;

import java.time.LocalDateTime;
import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import jsp.springboot.entity.Order;
import jsp.springboot.enums.OrderStatus;

public interface OrderRepository extends JpaRepository<Order, Integer> {
    // 4. Get Orders Of A Customer
    List<Order> findByCustomerCustomerId(Integer customerId);

    // 7. Get Orders By Status
    List<Order> findByStatus(OrderStatus status);

    // 8. Get Orders By Date
    List<Order> findByOrderDateTimeBetween(LocalDateTime start, LocalDateTime end);

    // 9. Get Orders Between Total Amount Range
    List<Order> findByTotalAmountBetween(Double min, Double max);

    // 10. Get Orders Of A Particular Restaurant
    List<Order> findByOrderItemsItemRestaurantRestaurantId(Integer restaurantId);
}
```

GlobalExceptionHandler.java

```
// PaymentException
@ExceptionHandler(PaymentException.class)
public ResponseEntity<ResponseStructure<String>> handlePaymentException(PaymentException exception) {

    ResponseStructure<String> response = new ResponseStructure<>();

    response.setStatusCode(HttpStatus.BAD_REQUEST.value());
    response.setMessage(exception.getMessage());
    response.setData("Failure");

    return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.BAD_REQUEST);
}
```

PaymentException.java

```
package jsp.springboot.exception;

public class PaymentException extends RuntimeException{
    public PaymentException(String message) {
        super(message);
    }
}
```

GlobalExceptionHandler.java

```
// OrderStateException
@ExceptionHandler(OrderStateException.class)
public ResponseEntity<ResponseStructure<String>> handleOrderStateException(OrderStateException exception) {

    ResponseStructure<String> response = new ResponseStructure<>();

    response.setStatusCode(HttpStatus.BAD_REQUEST.value());
    response.setMessage(exception.getMessage());
    response.setData("Failure");

    return new ResponseEntity<ResponseStructure<String>>(response, HttpStatus.BAD_REQUEST);
}
```

OrderStateException.java

```
package jsp.springboot.exception;

public class OrderStateException extends RuntimeException{
    public OrderStateException(String message) {
        super(message);
    }
}
```

Enum -> **OrderStatus**

```
package jsp.springboot.enums;

public enum OrderStatus {
    PLACED,
    PREPARING,
    DELIVERED,
    CANCELLED
}
```

PaymentMethod

```
package jsp.springboot.enums;

public enum PaymentMethod {
    CASH,
    CARD,
    UPI
}
```

PaymentStatus

```
package jsp.springboot.enums;

public enum PaymentStatus {
    PENDING,
    SUCCESS,
    FAILED
}
```

OrderService.java

```
package jsp.springboot.service;

import java.time.LocalDateTime;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.*;
import org.springframework.stereotype.Service;
import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.*;
import jsp.springboot.enums.*;
import jsp.springboot.exception.*;
import jsp.springboot.repository.*;

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private CustomerRepository customerRepository;

    @Autowired
    private MenuItemRepository menuItemRepository;
```

```java
// ================================
// 1. PLACE ORDER
// ================================
public ResponseEntity<ResponseStructure<Order>> placeOrder(Order order) {

    //  Customer validation
    if (order.getCustomer() == null)
        throw new InvalidInputException("Customer is required");

    Customer customer = customerRepository.findById(
            order.getCustomer().getCustomerId())
            .orElseThrow(() -> new ResourceNotFoundException("Customer not found"));

    // Order items validation
    List<OrderItem> items = order.getOrderItems();
    if (items == null || items.isEmpty())
        throw new InvalidInputException("Order must have at least one item");

    double total = 0;

    for (OrderItem item : items) {

        if (item.getQuantity() < 1)
            throw new InvalidInputException("Quantity must be minimum 1");

        Item menuItem = menuItemRepository.findById(
                item.getItem().getItemId())
                .orElseThrow(() -> new ResourceNotFoundException("Item not found"));

        if (!menuItem.isAvailable())
            throw new BusinessValidationException("Item not available");

        double subTotal = menuItem.getPrice() * item.getQuantity();

        item.setSubTotal(subTotal);
        item.setItem(menuItem);
        item.setOrder(order);

        total += subTotal;
    }
```

```java
        //  Payment validation
        Payment payment = order.getPayment();
        if (payment == null)
            throw new PaymentException("Payment is required");

        if (payment.getPaymentMethod() == null) {
            payment.setPaymentMethod(PaymentMethod.UPI);
        }

        if (payment.getPaymentStatus() == null) {
            payment.setPaymentStatus(PaymentStatus.SUCCESS);
        }

        payment.setAmount(total);
        payment.setOrder(order);

        // Order setup
        order.setCustomer(customer);
        order.setTotalAmount(total);
        order.setOrderDateTime(LocalDateTime.now());

        if (order.getStatus() == null)
            order.setStatus(OrderStatus.PLACED);

        // Save
        Order savedOrder = orderRepository.save(order);

        // Response
        ResponseStructure<Order> response = new ResponseStructure<>();
        response.setStatusCode(HttpStatus.CREATED.value());
        response.setMessage("Order Placed Successfully");
        response.setData(savedOrder);

        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }
```

OrderController.java

```java
package jsp.springboot.controller;

import java.time.LocalDateTime;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Order;
import jsp.springboot.enums.OrderStatus;
import jsp.springboot.service.OrderService;

@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    private OrderService service;

    // 1. PLACE ORDER
    @PostMapping
    public ResponseEntity<ResponseStructure<Order>> placeOrder(@RequestBody Order order) {
        return service.placeOrder(order);
    }
```

Output(Postman) :

POST http://localhost:8080/orders Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify


```
{
  "orderDateTime": "2026-04-28T10:30:00",
  "status": "PLACED",
  "totalAmount": 350.0,
  "customer": {
    "customerId": 1
  },
  "orderItems": [
    {
      "item": {
        "itemId": 1
      },
      "quantity": 2,
      "price": 175.0
    }
  ],
  "payment": {
    "paymentMethod": "UPI",
    "amount": 350.0,
    "paymentStatus": "SUCCESS"
  }
}
```


pgAdmin :

Data Output Messages Notifications

Showing rows: 1 to 6

	order_id [PK] integer	order_date_time timestamp without time zone (6)	status character varying (255)	total_amount double precision	customer_id integer
1	1	2026-04-28 02:02:38.524822	PLACED	240	1
2	2	2026-04-28 02:04:46.448556	PLACED	240	1
3	3	2026-04-28 02:06:11.420646	PREPARING	800	3
4	4	2026-04-28 02:06:35.804037	PLACED	320	6
5	5	2026-04-28 02:06:54.926715	PREPARING	660	8
6	6	2026-04-28 02:08:54.89878	DELIVERED	480	11

2. Get All Orders(OrderService.java)

```java
// ================================
// 2. Get All Orders
// ================================
public ResponseEntity<ResponseStructure<List<Order>>> getAllOrders() {
    List<Order> orders = orderRepository.findAll();
    if (orders.isEmpty())
        throw new ResourceNotFoundException("No orders found");

    ResponseStructure<List<Order>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Orders fetched successfully");
    response.setData(orders);
    return new ResponseEntity<ResponseStructure<List<Order>>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 2. Get All Orders
@GetMapping
public ResponseEntity<ResponseStructure<List<Order>>> getAllOrders() {
    return service.getAllOrders();
}
```

Output(Postman) :

GET http://localhost:8080/orders Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 2.50 s 62.1 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "customer": {
                "address": "Belgaum, Karnataka",
                "contact": "9876543210",
                "customerId": 1,
                "customerName": "Ganesh",
                "email": "ganesh1@example.com"
            },
            "orderDateTime": "2026-04-28T02:02:38.524822",
            "orderId": 1,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 1,
                        "itemName": "Veg Burger",
                        "price": 120.0,
                        "restaurant": {
                            "location": "Bangalore",
                            "restaurantId": 1,
                            "restaurantName": "Spicy Delight"
                        }
                    },
                    "order": {
                        "customer": {
```

3. Get Order By ID(OrderService.java)

```java
// ==============================
// 3. Get Order By ID
// ==============================
public ResponseEntity<ResponseStructure<Order>> getOrderById(Integer id) {
    Order order = orderRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Order not found"));

    ResponseStructure<Order> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Order fetched successfully");
    response.setData(order);
    return new ResponseEntity<ResponseStructure<Order>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 3. Get Order By ID
@GetMapping("/{id}")
public ResponseEntity<ResponseStructure<Order>> getOrderById(@PathVariable Integer id) {
    return service.getOrderById(id);
}
```

Output(Postman) :

GET http://localhost:8080/orders/3 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 71 ms 61.21 KB

{} JSON Preview Visualize


```
{
    "data": {
        "customer": {
            "address": "Mysore, Karnataka",
            "contact": "9876543212",
            "customerId": 3,
            "customerName": "Sneha",
            "email": "sneha3@example.com"
        },
        "orderDateTime": "2026-04-28T02:06:11.420646",
        "orderId": 3,
        "orderItems": [
            {
                "item": {
                    "available": true,
                    "itemId": 3,
                    "itemName": "Masala Dosa",
                    "price": 200.0,
                    "restaurant": {
                        "location": "Hyderabad",
                        "restaurantId": 3,
                        "restaurantName": "Taste Hub"
                    }
                },
                "order": {
                    "customer": {
                        "address": "Mysore, Karnataka",
```

4. Get Orders Of A Customer(OrderService.java)

```java
// ==============================
// 4. Get Orders Of A Customer
// ==============================
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByCustomer(Integer customerId) {
    List<Order> orders = orderRepository.findByCustomerCustomerId(customerId);
    if (orders.isEmpty())
        throw new ResourceNotFoundException("No orders found for this customer");

    ResponseStructure<List<Order>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Customer orders fetched");
    response.setData(orders);

    return new ResponseEntity<ResponseStructure<List<Order>>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 4. Get Orders Of A Customer
@GetMapping("/customer/{customerId}")
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByCustomer(@PathVariable Integer customerId) {
    return service.getOrdersByCustomer(customerId);
}
```

Output(Postman) :

GET http://localhost:8080/orders/customer/1 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 78 ms 62.1 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "customer": {
                "address": "Belgaum, Karnataka",
                "contact": "9876543210",
                "customerId": 1,
                "customerName": "Ganesh",
                "email": "ganesh1@example.com"
            },
            "orderDateTime": "2026-04-28T02:02:38.524822",
            "orderId": 1,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 1,
                        "itemName": "Veg Burger",
                        "price": 120.0,
                        "restaurant": {
                            "location": "Bangalore",
                            "restaurantId": 1,
                            "restaurantName": "Spicy Delight"
                        }
                    },
                    "order": {
                        "customer": {
```

5. Update Order Status(OrderService.java) :

```java
// ==============================
// 5. Update Order Status
// ==============================
public ResponseEntity<ResponseStructure<Order>> updateOrderStatus(Integer id, OrderStatus status) {
    Order order = orderRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Order not found"));

    order.setStatus(status);
    Order updatedOrder = orderRepository.save(order);

    ResponseStructure<Order> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Order status updated");
    response.setData(updatedOrder);

    return new ResponseEntity<ResponseStructure<Order>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 5. Update Order Status
@PutMapping("/{id}/status")
public ResponseEntity<ResponseStructure<Order>> updateOrderStatus(@PathVariable Integer id, @RequestParam OrderStatus status) {
    return service.updateOrderStatus(id, status);
}
```

Output(Postman) :

PUT http://localhost:8080/orders/7/status?status=PLACED Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

Body 200 OK 153 ms 62.99 KB

{} JSON Preview Visualize


```
{
    "data": {
        "customer": {
            "address": "Bijapur, Karnataka",
            "contact": "9876543217",
            "customerId": 8,
            "customerName": "Vikas",
            "email": "vikas8@example.com"
        },
        "orderDateTime": "2026-04-28T13:08:16.555008",
        "orderId": 7,
        "orderItems": [
            {
                "item": {
                    "available": true,
                    "itemId": 8,
                    "itemName": "Paneer Butter Masala",
                    "price": 220.0,
                    "restaurant": {
                        "location": "Mumbai",
                        "restaurantId": 5,
                        "restaurantName": "Royal Kitchen"
                    }
```

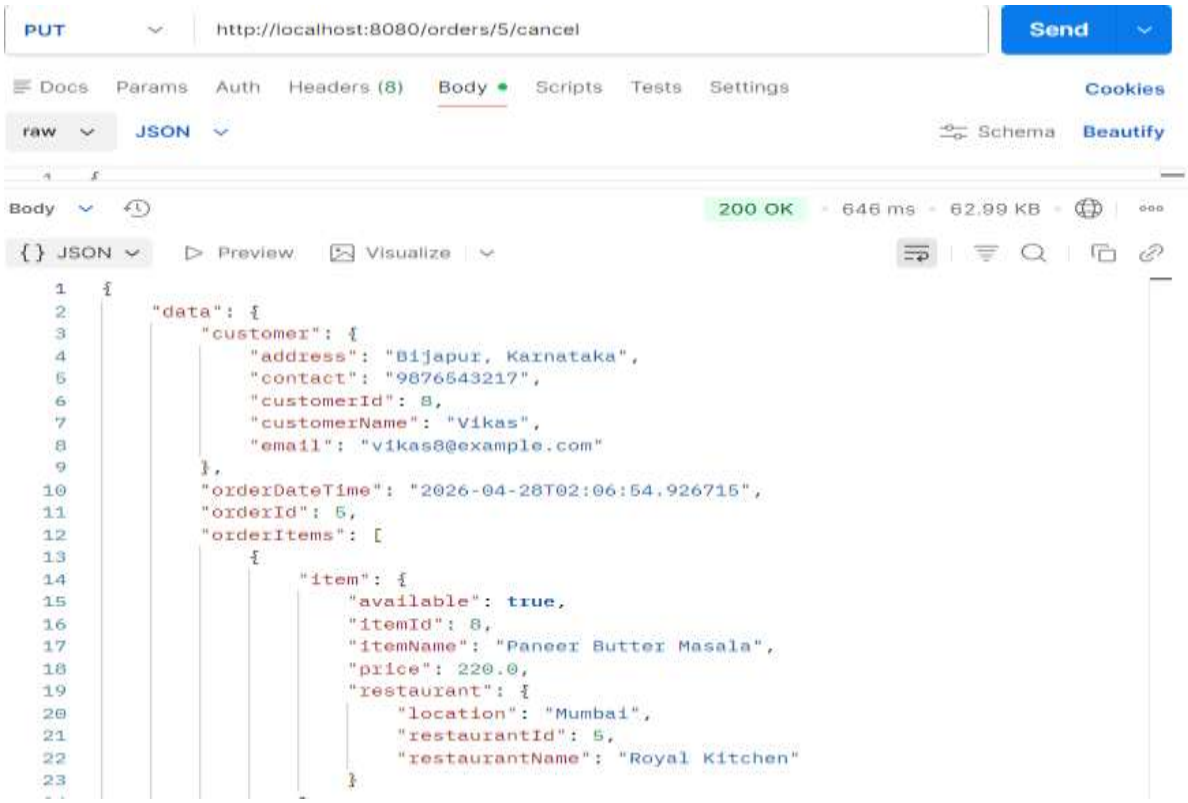
6. Cancel Order(OrderService.java) :

```java
// ==============================
// 6. Cancel Order
// ==============================
public ResponseEntity<ResponseStructure<Order>> cancelOrder(Integer id) {
    Order order = orderRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Order not found"));

    if (order.getStatus() == OrderStatus.DELIVERED)
        throw new OrderStateException("Delivered order cannot be cancelled");

    order.setStatus(OrderStatus.CANCELLED);
    Order updatedOrder = orderRepository.save(order);

    ResponseStructure<Order> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Order cancelled successfully");
    response.setData(updatedOrder);

    return new ResponseEntity<ResponseStructure<Order>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 6. Cancel Order
@PutMapping("/{id}/cancel")
public ResponseEntity<ResponseStructure<Order>> cancelOrder(@PathVariable Integer id) {
    return service.cancelOrder(id);
}
```

Output(Postman) :



7. Get Orders By Status(OrderService.java) :

```java
// ================================
// 7. Get Orders By Status
// ================================
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByStatus(OrderStatus status) {
    List<Order> orders = orderRepository.findByStatus(status);
    if (orders.isEmpty())
        throw new ResourceNotFoundException("No orders found");

    ResponseStructure<List<Order>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Orders fetched by status");
    response.setData(orders);
    return new ResponseEntity<ResponseStructure<List<Order>>>(response, HttpStatus.OK);
}
```

OrderController.java

```java
// 7. Get Orders By Status
// http://localhost:8080/orders/status?status=PLACED
@GetMapping("/status")
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByStatus(@RequestParam OrderStatus status) {
    return service.getOrdersByStatus(status);
}
```

Output(Postman) :

GET http://localhost:8080/orders/status?status=PLACED Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

Body 200 OK 830 ms 62.1 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "customer": {
                "address": "Belgaum, Karnataka",
                "contact": "9876543210",
                "customerId": 1,
                "customerName": "Ganesh",
                "email": "ganesh1@example.com"
            },
            "orderDateTime": "2026-04-28T02:02:38.524822",
            "orderId": 1,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 1,
                        "itemName": "Veg Burger",
                        "price": 120.0,
                        "restaurant": {
                            "location": "Bangalore",
                            "restaurantId": 1,
                            "restaurantName": "Spicy Delight"
```

8. Get Orders By Date(OrderService.java) :

```java
// ==============================
// 8. Get Orders By Date
// ==============================
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByDate(LocalDateTime start, LocalDateTime end) {
    List<Order> orders = orderRepository.findByOrderDateTimeBetween(start, end);

    if (orders.isEmpty())
        throw new ResourceNotFoundException("No orders found");
    ResponseStructure<List<Order>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Orders fetched by date");
    response.setData(orders);

    return new ResponseEntity<ResponseStructure<List<Order>>>(response, HttpStatus.OK);
}
```

OrderController.java

```java
// 8. Get Orders By Date
// http://localhost:8080/orders/date?start=2026-03-20T00:00:00&end=2026-03-23T23:59:59
@GetMapping("/date")
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByDate(@RequestParam LocalDateTime start, @RequestParam LocalDateTime end) {
    return service.getOrdersByDate(start, end);
}
```

Output(Postman) :

GET http://localhost:8080/orders/date?start=2026-04-01T00:00:00&end=2026-... Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

Body 200 OK 326 ms 62.1 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "customer": {
                "address": "Belgaum, Karnataka",
                "contact": "9876543210",
                "customerId": 1,
                "customerName": "Ganesh",
                "email": "ganesh1@example.com"
            },
            "orderDateTime": "2026-04-28T02:02:38.524822",
            "orderId": 1,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 1,
                        "itemName": "Veg Burger",
                        "price": 120.0,
                        "restaurant": {
                            "location": "Bangalore",
                            "restaurantId": 1,
                            "restaurantName": "Spicy Delight"
```

9. Get Orders Between Total Amount Range(OrderService.java) :

```java
// ==============================
// 9. Get Orders Between Total Amount Range
// ==============================
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByAmount(Double min, Double max) {
    List<Order> orders = orderRepository.findByTotalAmountBetween(min, max);

    if (orders.isEmpty())
        throw new ResourceNotFoundException("No orders found");

    ResponseStructure<List<Order>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Orders fetched by amount range");
    response.setData(orders);
    return new ResponseEntity<ResponseStructure<List<Order>>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 9. Get Orders Between Total Amount Range
//http://localhost:8080/orders/amount?min=100&max=500
@GetMapping("/amount")
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByAmount(@RequestParam Double min, @RequestParam Double max) {
    return service.getOrdersByAmount(min, max);
}
```

Output(Postman) :

GET http://localhost:8080/orders/amount?min=400&max=500 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

Body 200 OK 40 ms 62.1 KB

{} JSON Preview Visualize


```
1  {
2      "data": [
3          {
4              "customer": {
5                  "address": "Udupi, Karnataka",
6                  "contact": "9879363218",
7                  "customerId": 11,
8                  "customerName": "Preeti Sahu",
9                  "email": "preeti@gmail.com"
10             },
11             "orderDateTime": "2026-04-28T02:08:54.89878",
12             "orderId": 6,
13             "orderItems": [
14                 {
15                     "item": {
16                         "available": true,
17                         "itemId": 1,
18                         "itemName": "Veg Burger",
19                         "price": 120.0,
20                         "restaurant": {
21                             "location": "Bangalore",
22                             "restaurantId": 1,
23                             "restaurantName": "Spicy Delight"
```

10. Get Orders Of A Particular Restaurant(OrderService.java) :

```java
// ================================
// 10. Get Orders Of A Particular Restaurant
// ================================
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByRestaurant(Integer restaurantId) {
    List<Order> orders = orderRepository.findByOrderItemsItemRestaurantRestaurantId(restaurantId);

    if (orders.isEmpty())
        throw new ResourceNotFoundException("No orders found for this restaurant");

    ResponseStructure<List<Order>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Restaurant orders fetched");
    response.setData(orders);

    return new ResponseEntity<ResponseStructure<List<Order>>>(response, HttpStatus.OK);
}
```

<u>OrderController.java</u>

```java
// 10. Get Orders Of A Particular Restaurant
@GetMapping("/restaurant/{restaurantId}")
public ResponseEntity<ResponseStructure<List<Order>>> getOrdersByRestaurant(@PathVariable Integer restaurantId) {
    return service.getOrdersByRestaurant(restaurantId);
}
```

Output(Postman) :

GET http://localhost:8080/orders/restaurant/5 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

Body 200 OK 401 ms 62.9 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "customer": {
                "address": "Bijapur, Karnataka",
                "contact": "9876543217",
                "customerId": 8,
                "customerName": "Vikas",
                "email": "vikas8@example.com"
            },
            "orderDateTime": "2026-04-28T02:06:54.926715",
            "orderId": 5,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 8,
                        "itemName": "Paneer Butter Masala",
                        "price": 220.0,
                        "restaurant": {
                            "location": "Mumbai",
                            "restaurantId": 5,
                            "restaurantName": "Royal Kitchen"
```

8.6 Order Item :

OrderItem.java

```
package jsp.springboot.entity;

import jakarta.persistence.*;

@Entity
public class OrderItem {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer orderItemId;
    private Integer quantity;
    private Double subTotal;

    @ManyToOne
    @JoinColumn(name="order_id")
    private Order order;

    @ManyToOne
    @JoinColumn(name="item_id")
    private Item item;

    // getter and setter


}
```

OrderItemRepository.java

```
package jsp.springboot.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import jsp.springboot.entity.OrderItem;

public interface OrderItemRepository extends JpaRepository<OrderItem, Integer>{
    // 4. Get all order items of an order
    List<OrderItem> findByOrderOrderId(Integer orderId);
}
```

OrderItemService.java

```java
package jsp.springboot.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.*;
import org.springframework.stereotype.Service;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.*;
import jsp.springboot.exception.*;
import jsp.springboot.repository.*;

@Service
public class OrderItemService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private MenuItemRepository menuItemRepository;

    @Autowired
    private OrderItemRepository orderItemRepository;
    // 1. Add Item to Existing Order
    public ResponseEntity<ResponseStructure<Order>> addItemToOrder(int orderId, OrderItem newItem) {

        // Order must exist
        Order order = orderRepository.findById(orderId)
                .orElseThrow(() -> new ResourceNotFoundException("Order not found"));

        //  Quantity must be >= 1
        if (newItem.getQuantity() < 1)
            throw new InvalidInputException("Quantity must be minimum 1");

        //  Fetch item from DB
        Item menuItem = menuItemRepository.findById(
                newItem.getItem().getItemId())
                .orElseThrow(() -> new ResourceNotFoundException("Item not found"));
```

```java
        // Item must be available
        if (!menuItem.isAvailable())
            throw new BusinessValidationException("Item not available");

        //  Subtotal = price * quantity
        double subTotal = menuItem.getPrice() * newItem.getQuantity();

        newItem.setSubTotal(subTotal);
        newItem.setItem(menuItem);
        newItem.setOrder(order);

        //  Add item to order
        List<OrderItem> items = order.getOrderItems();
        items.add(newItem);

        //  Recalculate total
        double total = 0;
        for (OrderItem item : items) {
            total += item.getSubTotal();
        }
        order.setTotalAmount(total);

        // Update payment
        if (order.getPayment() != null) {
            order.getPayment().setAmount(total);
        }


        ResponseStructure<Order> response = new ResponseStructure<>();
        response.setStatusCode(HttpStatus.CREATED.value());
        response.setMessage("Item added successfully");
        response.setData(orderRepository.save(order));

        return new ResponseEntity<ResponseStructure<Order>>(response, HttpStatus.CREATED);
    }
```

OrderItemController.java

```java
package jsp.springboot.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.*;
import jsp.springboot.service.OrderItemService;

@RestController
@RequestMapping("/ordersItem")
public class OrderItemController {
    @Autowired
    private OrderItemService service;

    // 1. add item to existing order
    @PostMapping("/{orderId}/items")
    public ResponseEntity<ResponseStructure<Order>> addItemToOrder(@PathVariable Integer orderId, @RequestBody OrderItem item) {
        return service.addItemToOrder(orderId, item);
    }
```

Output(Postman) :

POST http://localhost:8080/ordersItem/1/items Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
{
  "quantity": 2,
  "subTotal": 240.0,
  "order": {
    "orderId": 1
  },
  "item": {
    "itemId": 3
  }
}
```

Body 201 Created 481 ms 62.18 KB

JSON Preview Visualize


```
{
    "data": {
        "customer": {
            "address": "Belgaum, Karnataka",
            "contact": "9876543210",
            "customerId": 1,
            "customerName": "Ganesh",
            "email": "ganesh1@example.com"
        },
        "orderDateTime": "2026-04-28T02:02:38.524822",
        "orderId": 1,
        "orderItems": [
            {
```


pgAdmin :

Data Output Messages Notifications

	order_item_id [PK] integer	quantity integer	sub_total double precision	item_id integer	order_id integer
1	1	2	240	1	1
2	2	2	240	1	2
3	3	4	800	3	3
4	4	2	320	6	4
5	5	3	660	8	5
6	6	4	480	1	6
7	7	3	660	8	7
8	8	2	400	3	1

2. Update order item quantity(OrderItemService.java) :

```java
// 2. Update item quantity
public ResponseEntity<ResponseStructure<OrderItem>> updateOrderItemQuantity(Integer orderItemId, Integer quantity){
    if(quantity < 1)
        throw new InvalidInputException("Quantity must be minimum 1");

    OrderItem orderItem = orderItemRepository.findById(orderItemId)
            .orElseThrow(() -> new ResourceNotFoundException("OrderItem not found"));
    orderItem.setQuantity(quantity);

    double subTotal = orderItem.getItem().getPrice() * quantity;
    orderItem.setSubTotal(subTotal);
    Order order = orderItem.getOrder();

    double total = 0;
    for(OrderItem item : order.getOrderItems()){
        total += item.getSubTotal();
    }

    order.setTotalAmount(total);
    orderRepository.save(order);

    ResponseStructure<OrderItem> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Order item quantity updated");
    response.setData(orderItem);

    return new ResponseEntity<ResponseStructure<OrderItem>>(response,HttpStatus.OK);
}
```

<u>OrderItemController.java</u>

```java
// 2. Update order item quantity
@PutMapping("/update/{orderItemId}/{quantity}")
public ResponseEntity<ResponseStructure<OrderItem>> updateQuantity(@PathVariable Integer orderItemId, @PathVariable Integer quantity){
    return service.updateOrderItemQuantity(orderItemId, quantity);
}
```

Output(Postman) :

PUT http://localhost:8080/ordersItem/update/1/5 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 775 ms 62.27 KB

{} JSON Preview Visualize


```
1  {
2      "data": {
3          "item": {
4              "available": true,
5              "itemId": 1,
6              "itemName": "Veg Burger",
7              "price": 120.0,
8              "restaurant": {
9                  "location": "Bangalore",
10                 "restaurantId": 1,
11                 "restaurantName": "Spicy Delight"
12             }
13         },
```

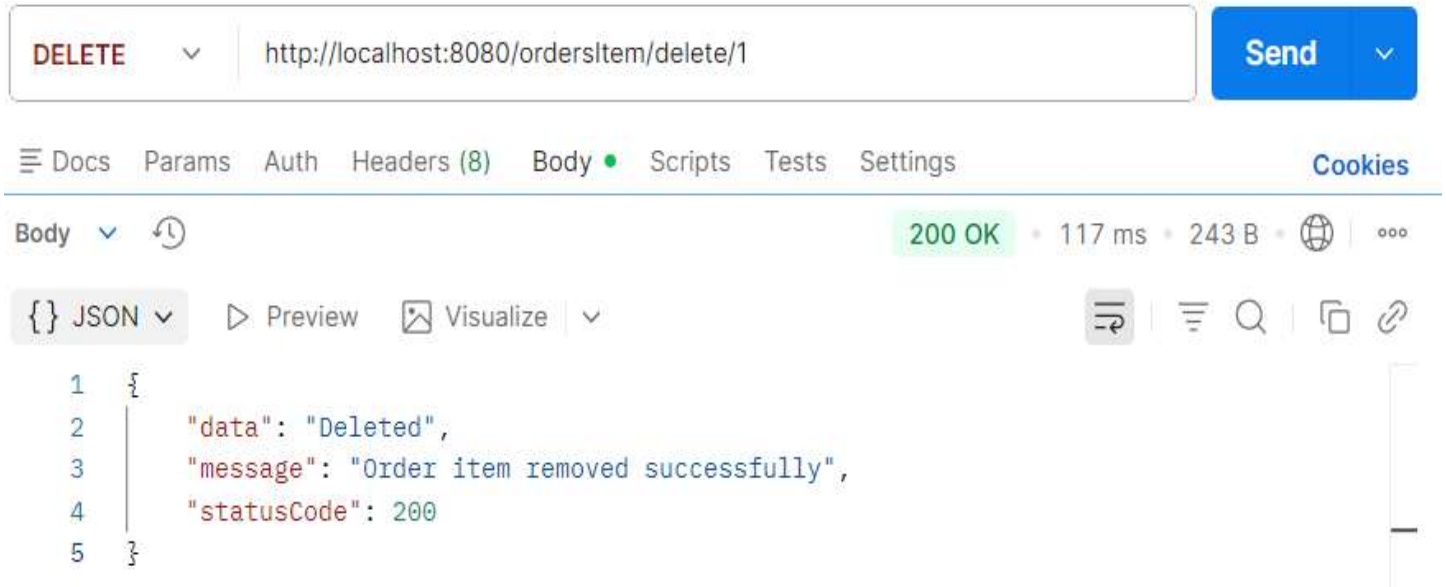
3. Remove order item(OrderItemService.java) :

```java
// 3. Remove item from order
public ResponseEntity<ResponseStructure<String>> removeOrderItem(Integer orderItemId){
    OrderItem orderItem = orderItemRepository.findById(orderItemId)
            .orElseThrow(() -> new ResourceNotFoundException("OrderItem not found"));

    Order order = orderItem.getOrder();
    order.getOrderItems().remove(orderItem);
    orderItemRepository.delete(orderItem);

    double total = 0;
    for(OrderItem item : order.getOrderItems()){
        total += item.getSubTotal();
    }

    order.setTotalAmount(total);
    orderRepository.save(order);

    ResponseStructure<String> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Order item removed successfully");
    response.setData("Deleted");

    return new ResponseEntity<ResponseStructure<String>>(response,HttpStatus.OK);
}
```

<u>OrderItemController.java</u>

```java
// 3. Remove order item
@DeleteMapping("/delete/{orderItemId}")
public ResponseEntity<ResponseStructure<String>> deleteOrderItem(@PathVariable Integer orderItemId){
    return service.removeOrderItem(orderItemId);
}
```

Output(Postman) :



4. Get order items of an order(OrderItemService.java) :

```java
// 4. Get order items of an order
public ResponseEntity<ResponseStructure<List<OrderItem>>> getOrderItemsByOrder(Integer orderId){
    Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new ResourceNotFoundException("Order not found"));

    List<OrderItem> items = order.getOrderItems();

    ResponseStructure<List<OrderItem>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Order items fetched successfully");
    response.setData(items);

    return new ResponseEntity<ResponseStructure<List<OrderItem>>>(response,HttpStatus.OK);
}
```

<u>OrderItemController.java</u>

```java
// 4. Get order items of an order
@GetMapping("/order/{orderId}")
public ResponseEntity<ResponseStructure<List<OrderItem>>> getItemsByOrder(@PathVariable Integer orderId){
    return service.getOrderItemsByOrder(orderId);
}
```

Output(Postman) :

GET http://localhost:8080/ordersItem/order/1 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 34 ms 61.77 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "item": {
                "available": true,
                "itemId": 3,
                "itemName": "Masala Dosa",
                "price": 200.0,
                "restaurant": {
                    "location": "Hyderabad",
                    "restaurantId": 3,
                    "restaurantName": "Taste Hub"
                }
            },
            "order": {
                "customer": {
                    "address": "Belgaum, Karnataka",
                    "contact": "9876543210",
                    "customerId": 1,
                    "customerName": "Ganesh",
                    "email": "ganesh1@example.com"
                },
                "orderDateTime": "2026-04-28T02:02:38.524822",
                "orderId": 1,
                "orderItems": [
                    {
```

8.7 Payment

Payment.java

```java
package jsp.springboot.entity;

import jakarta.persistence.*;
import jsp.springboot.enums.*;

@Entity
public class Payment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer paymentyId;

    @Enumerated(EnumType.STRING)
    private PaymentMethod paymentMethod;

    @Enumerated(EnumType.STRING)
    private PaymentStatus paymentStatus;

    private Double amount;

    @OneToOne
    @JoinColumn(name = "order_id")
    private Order order;

    // getter and setter
}
```

PaymentRepository.java

```java
package jsp.springboot.repository;

import java.util.List;
import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import jsp.springboot.entity.Payment;
import jsp.springboot.enums.PaymentMethod;
import jsp.springboot.enums.PaymentStatus;

public interface PaymentRepository extends JpaRepository<Payment, Integer> {
    // 2. Get payment by status
    List<Payment> findByPaymentStatus(PaymentStatus paymentStatus);

    // 3. Get payment by payment method
    List<Payment> findByPaymentMethod(PaymentMethod paymentMethod);
}
```

PaymentService.java

```java
package jsp.springboot.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.*;
import org.springframework.stereotype.Service;
import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.*;
import jsp.springboot.enums.*;
import jsp.springboot.exception.*;
import jsp.springboot.repository.PaymentRepository;

@Service
public class PaymentService {

    @Autowired
    private PaymentRepository paymentRepository;

    // 1. Get Payment by ID
    public ResponseEntity<ResponseStructure<Payment>> getPaymentById(int paymentId) {

        // Check payment exists
        Payment payment = paymentRepository.findById(paymentId)
                .orElseThrow(() -> new ResourceNotFoundException("Payment not found"));

        //  Check order linked
        Order order = payment.getOrder();
        if (order == null)
            throw new BusinessValidationException("Payment is not linked to any order");

        //  Validate amount
        if (Math.abs(payment.getAmount() - order.getTotalAmount()) > 0.01) {
            throw new BusinessValidationException("Payment amount mismatch with order total");
        }

        ResponseStructure<Payment> response = new ResponseStructure<>();
        response.setStatusCode(HttpStatus.CREATED.value());
        response.setMessage("Payment fetched successfully");
        response.setData(payment);
        return new ResponseEntity<ResponseStructure<Payment>>(response, HttpStatus.CREATED);
    }
```

PaymentController.java

```java
package jsp.springboot.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jsp.springboot.dto.ResponseStructure;
import jsp.springboot.entity.Payment;
import jsp.springboot.enums.*;
import jsp.springboot.service.PaymentService;

@RestController
@RequestMapping("/payments")
public class PaymentController {

    @Autowired
    private PaymentService service;

    // 1. Get Payment by ID
    @GetMapping("/{paymentId}")
    public ResponseEntity<ResponseStructure<Payment>> getPaymentById(@PathVariable int paymentId) {
        return service.getPaymentById(paymentId);
    }
```

Output(Postman) :

GET http://localhost:8080/payments/5 Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 201 Created 797 ms 62.93 KB

{} JSON Preview Visualize


```
{
    "data": {
        "amount": 660.0,
        "order": {
            "customer": {
                "address": "Bijapur, Karnataka",
                "contact": "9876543217",
                "customerId": 8,
                "customerName": "Vikas",
                "email": "vikas8@example.com"
            },
            "orderDateTime": "2026-04-28T02:06:54.926715",
            "orderId": 5,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 8,
                        "itemName": "Paneer Butter Masala",
                        "price": 220.0,
                        "restaurant": {
                            "location": "Mumbai",
                            "restaurantId": 5,
                            "restaurantName": "Royal Kitchen"
                        }
                    },
                    "order": {
```


pgAdmin :

	payment_id [PK] integer	amount double precision	payment_method character varying (255)	payment_status character varying (255)	order_id integer
1	1	640	UPI	SUCCESS	1
2	2	240	UPI	SUCCESS	2
3	3	800	CASH	PENDING	3
4	4	320	CARD	FAILED	4
5	5	660	CASH	PENDING	5
6	6	480	UPI	SUCCESS	6
7	7	660	CASH	PENDING	7

2. Get Payment by Status(PaymentService.java) :

```java
// 2. Get Payment by Status
public ResponseEntity<ResponseStructure<List<Payment>>> getPaymentByStatus(PaymentStatus status) {
    List<Payment> payments = paymentRepository.findByPaymentStatus(status);
    if (payments.isEmpty()) {
        throw new ResourceNotFoundException("No payments found with status: " + status);
    }

    ResponseStructure<List<Payment>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Payments fetched by status successfully");
    response.setData(payments);

    return new ResponseEntity<ResponseStructure<List<Payment>>>(response, HttpStatus.OK);
}
```

PaymentController.java

```java
// 2. Get Payment by Status
@GetMapping("/status/{status}")
public ResponseEntity<ResponseStructure<List<Payment>>> getPaymentByStatus(@PathVariable PaymentStatus status) {
    return service.getPaymentByStatus(status);
}
```

Output(Postman) :

GET http://localhost:8080/payments/status/PENDING Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 623 ms 61.14 KB

{} JSON Preview Visualize


```
{
    "data": [
        {
            "amount": 800.0,
            "order": {
                "customer": {
                    "address": "Mysore, Karnataka",
                    "contact": "9876543212",
                    "customerId": 3,
                    "customerName": "Sneha",
                    "email": "sneha3@example.com"
                },
                "orderDateTime": "2026-04-28T02:06:11.420646",
                "orderId": 3,
                "orderItems": [
                    {
                        "item": {
                            "available": true,
                            "itemId": 3,
                            "itemName": "Masala Dosa",
                            "price": 200.0,
                            "restaurant": {
                                "location": "Hyderabad",
                                "restaurantId": 3,
                                "restaurantName": "Taste Hub"
                            }
                        },
```

3. Get Payment by Payment Method(PaymentService.java) :

```java
// 3. Get Payment by Payment Method
public ResponseEntity<ResponseStructure<List<Payment>>> getPaymentByMethod(PaymentMethod method) {
    List<Payment> payments = paymentRepository.findByPaymentMethod(method);

    if (payments.isEmpty()) {
        throw new ResourceNotFoundException("No payments found with method: " + method);
    }

    ResponseStructure<List<Payment>> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Payments fetched by payment method successfully");
    response.setData(payments);

    return new ResponseEntity<ResponseStructure<List<Payment>>>(response, HttpStatus.OK);
}
```

PaymentController.java

```java
// 3. Get Payment by Payment Method
@GetMapping("/method/{method}")
public ResponseEntity<ResponseStructure<List<Payment>>> getPaymentByMethod(@PathVariable PaymentMethod method) {
    return service.getPaymentByMethod(method);
}
```

Output(Postman) :

GET http://localhost:8080/payments/method/CARD Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 103 ms 61.63 KB

{} JSON Preview Visualize


```
1  {
2      "data": [
3          {
4              "amount": 320.0,
5              "order": {
6                  "customer": {
7                      "address": "Dharwad, Karnataka",
8                      "contact": "9876543215",
9                      "customerId": 6,
10                     "customerName": "Kiran",
11                     "email": "kiran6@example.com"
12                 },
13                 "orderDateTime": "2026-04-28T02:06:35.804037",
14                 "orderId": 4,
15                 "orderItems": [
16                     {
17                         "item": {
18                             "available": true,
19                             "itemId": 6,
20                             "itemName": "Chole Bhature",
21                             "price": 160.0,
22                             "restaurant": {
23                                 "location": "Chennai",
24                                 "restaurantId": 2,
25                                 "restaurantName": "Food Palace"
26                             }
27                         },
```

4. Update Payment Status(PaymentService.java) :

```java
// 4. Update Payment Status
public ResponseEntity<ResponseStructure<Payment>> updatePaymentStatus(int paymentId, PaymentStatus status) {
    Payment payment = paymentRepository.findById(paymentId)
            .orElseThrow(() -> new ResourceNotFoundException("Payment not found"));

    payment.setPaymentStatus(status);
    Payment updatedPayment = paymentRepository.save(payment);

    ResponseStructure<Payment> response = new ResponseStructure<>();
    response.setStatusCode(HttpStatus.OK.value());
    response.setMessage("Payment status updated successfully");
    response.setData(updatedPayment);

    return new ResponseEntity<ResponseStructure<Payment>>(response, HttpStatus.OK);
}
```

PaymentController.java

```java
// 4. Update Payment Status
@PutMapping("/{paymentId}/{status}")
public ResponseEntity<ResponseStructure<Payment>> updatePaymentStatus(@PathVariable int paymentId, @PathVariable PaymentStatus status) {
    return service.updatePaymentStatus(paymentId, status);
}
```

Output(Postman) :

PUT http://localhost:8080/payments/3/FAILED Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookies

Body 200 OK 266 ms 61.15 KB

{} JSON Preview Visualize


```
{
    "data": {
        "amount": 800.0,
        "order": {
            "customer": {
                "address": "Mysore, Karnataka",
                "contact": "9876543212",
                "customerId": 3,
                "customerName": "Sneha",
                "email": "sneha3@example.com"
            },
            "orderDateTime": "2026-04-28T02:06:11.420646",
            "orderId": 3,
            "orderItems": [
                {
                    "item": {
                        "available": true,
                        "itemId": 3,
                        "itemName": "Masala Dosa",
                        "price": 200.0,
                        "restaurant": {
                            "location": "Hyderabad",
                            "restaurantId": 3,
                            "restaurantName": "Taste Hub"
                        }
                    },
```

9. Future Enhancements

The **Online Food Delivery Management System (FoodieHub)** has been designed as a scalable and flexible platform that can be continuously upgraded based on market trends, customer expectations, and technological advancements. Although the current version provides essential features such as restaurant listing, menu management, order placement, payment handling, and delivery coordination, there are many opportunities for future enhancement. These improvements can make the system smarter, faster, more secure, and more user-friendly. The following are the major future enhancements planned for the project.

1. Mobile Application Development

One of the most important future enhancements is the development of dedicated mobile applications for **Android** and **iOS** platforms. While the web version is useful, mobile apps provide better convenience, faster access, push notifications, GPS integration, and improved customer experience. Customers can place orders anytime and anywhere using their smartphones.

2. Real-Time Order Tracking

Future versions can include **live order tracking** using GPS technology. Customers will be able to track their food order from restaurant preparation to final delivery. A map-based interface can show the current location of the delivery partner, estimated arrival time, and route updates. This feature improves transparency and customer satisfaction.

3. AI-Based Food Recommendations

Artificial Intelligence can be integrated to analyze customer preferences, previous orders, ratings, and browsing behavior. Based on this data, the system can recommend dishes, restaurants, combos, or offers that match user interests. Personalized recommendations can increase sales and customer engagement.

4. Voice Search and Voice Ordering

Modern systems can support **voice assistants** where customers search restaurants or place orders using voice commands. This feature improves accessibility for elderly users and provides hands-free convenience.

5. Chatbot Customer Support

An AI-powered chatbot can be added to provide instant customer support 24/7. It can answer common questions such as order status, refund policy, payment issues, restaurant timings, and complaint registration. This reduces human workload and improves response speed.

6. Advanced Payment Gateway Integration

Currently basic payment methods may be supported, but future enhancement can include multiple secure payment options such as:

- UPI
- Credit/Debit Cards
- Net Banking
- Digital Wallets
- EMI Payments
- Buy Now Pay Later

Secure payment encryption and fraud detection mechanisms can also be added.

7. Subscription Membership Plans

The system can introduce premium membership programs similar to modern food delivery platforms. Members can receive:

- Free delivery
- Priority support
- Exclusive discounts
- Faster delivery options
- Reward points

This helps increase customer loyalty and recurring revenue.

8. Loyalty and Reward System

Customers can earn reward points on every purchase. These points can be redeemed for discounts, cashback, or free meals. A reward system motivates repeat orders and improves retention.

9. Smart Delivery Allocation

Machine learning algorithms can assign orders automatically to the nearest available delivery partner based on traffic, distance, and workload. This can reduce delivery time and optimize logistics operations.

10. Delivery Partner Mobile Panel

A separate application can be developed for delivery executives where they can:

- Accept or reject orders
- Navigate through maps
- Update delivery status
- View earnings
- Manage schedules
- Contact customers securely

This improves delivery management efficiency.

11. Restaurant Analytics Dashboard

Restaurant owners can receive detailed analytics reports including:

- Daily sales
- Popular dishes
- Peak order timings
- Customer feedback
- Revenue trends

- Cancelled orders

These insights help restaurants make better business decisions.

12. Inventory Management System

Future versions can connect restaurant inventory with menu items. If ingredients are out of stock, items can automatically become unavailable. This avoids customer disappointment and improves menu accuracy.

13. Multi-Vendor Expansion

The platform can be expanded into a large marketplace where thousands of restaurants can register independently, upload menus, manage prices, and receive orders. This converts the project into a complete commercial ecosystem.

14. Multilingual Support

The application can support multiple regional and international languages such as:

- English
- Tamil
- Hindi
- Telugu
- Kannada
- Malayalam

This allows wider customer reach and better usability.

15. Dark Mode and UI Personalization

Customers can customize themes, font sizes, colors, and accessibility settings. Dark mode can reduce eye strain and improve user comfort.

16. Scheduled Orders

Users can place orders in advance for future delivery such as lunch meetings, birthday parties, or office events. This adds convenience and improves planning.

17. Group Ordering Feature

A group ordering system can allow multiple users to add items to a shared cart for office teams, families, or friends. One user can make the final payment.

18. Contactless Delivery

Customers can choose contactless delivery where food is placed at the doorstep. This became highly important after pandemic situations and improves safety.

19. Rating and Review System Enhancement

Advanced review systems can include:

- Food quality rating

- Delivery speed rating
- Packaging rating
- Hygiene rating
- Image uploads with reviews

This helps users make informed choices.

20. Fraud Detection and Security

Future versions can use AI to detect fake orders, suspicious payments, coupon abuse, and spam accounts. Multi-factor authentication can be added for stronger security.

21. Smart Notifications

Users can receive personalized notifications for:

- Order updates
- Special offers
- Birthday discounts
- Favorite restaurant promotions
- Reorder reminders

Push notifications can improve customer engagement.

22. Cloud Deployment

The project can be deployed fully on cloud platforms such as AWS, Azure, or Google Cloud for better scalability, backup, security, and high availability.

23. API Integration with Third Parties

The system can integrate with:

- Google Maps
- SMS Gateways
- WhatsApp Notifications
- Payment Gateways
- CRM Systems
- ERP Solutions

This increases functionality and business automation.

24. Green Delivery Initiatives

Eco-friendly future enhancements may include:

- Electric vehicle deliveries
- Carbon footprint tracking

- Paperless bills
- Eco packaging preference options

This supports sustainability goals.

25. Augmented Reality Menu Preview

AR technology can allow users to preview dishes in 3D before ordering. Customers can visualize portion size and presentation through mobile cameras.

26. Health and Nutrition Filters

Users can filter food based on:

- Low calorie
- High protein
- Vegan
- Gluten free
- Diabetic friendly
- Keto meals

Nutrition data can also be shown with menu items.

27. Dynamic Pricing and Offers

The system can automatically generate offers based on demand, customer history, and restaurant campaigns. Examples include lunch discounts, weekend combos, and flash deals.

28. Social Media Integration

Customers can share favorite dishes, reviews, and referral links directly to social media platforms. Referral rewards can increase user acquisition.

29. Complaint and Refund Automation

An automated support system can process delayed orders, missing items, or wrong deliveries with instant refund workflows and complaint ticket generation.

30. Expansion to Grocery and Essentials Delivery

The same platform can later support:

- Grocery delivery
- Pharmacy delivery
- Fresh vegetables
- Bakery products
- Daily essentials

This creates a multi-service delivery ecosystem

10. Summary

The **Online Food Delivery Management System (FoodieHub)** is a modern digital platform developed to simplify and improve the process of ordering food online. It connects customers, restaurants, administrators, and delivery partners through a single integrated system. Customers can browse restaurants, explore menus, place orders, make payments, and track deliveries easily. Restaurants can manage menu items, update availability, process incoming orders, and monitor sales performance. Administrators can oversee users, restaurants, payments, and overall system activities efficiently.

The project was developed using modern technologies such as **Java, Spring Boot, Hibernate, MySQL, and REST APIs**, ensuring scalability, reliability, and secure data management. The system reduces manual work, increases operational efficiency, minimizes errors, and provides a smooth user experience. Features such as secure login, order management, payment processing, and delivery coordination make it a complete solution for food delivery businesses.

Future enhancements such as mobile applications, AI-based recommendations, real-time GPS tracking, chatbot support, cloud deployment, multilingual support, and analytics dashboards can further expand the capabilities of the platform. These upgrades can transform the system into a large-scale commercial application capable of serving a wider audience.

Overall, **FoodieHub** demonstrates how technology can modernize the food service industry by providing convenience to customers and better management tools for restaurants. It is a practical, scalable, and future-ready solution in the growing online food delivery market.